1 比赛 1

2 数学 1

3 数据结构 2

4 数值计算 5

5 数论 9

6 组合数学 11

7 图论 12

8 匹配 13

9 几何 16

10 字符串 19

11 杂项 21

比赛 (1)

template.cpp 9 lines

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll cdiv(ll a,ll b){return a/b+((a^b)>0&&a%b);} // 向上取整
ll fdiv(ll a,ll b){return a/b-((a^b)<0&&a%b);} // 向下取整
int main(){
  ios::sync_with_stdio(0);cin.tie(0);

}
```

hash.sh 3 lines

```sh
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

1.1 Python 速查

```python
import sys, math
from collections import deque, Counter, defaultdict
from heapq import heappush, heappop, heapify, nsmallest
from bisect import bisect_left, bisect_right, insort
from itertools import accumulate, combinations, product
from functools import lru_cache, cmp_to_key
from fractions import Fraction
from decimal import Decimal, getcontext
input = sys.stdin.readline  # 快速读入，行尾带换行
sys.setrecursionlimit(1 << 20)  # 递归深的题优先改成循环
n, m = map(int, input().split())
a = list(map(int, input().split()))
rest = sys.stdin.read().split()  # 一次读入剩下全部
print(*a)  # 空格分隔输出
sys.stdout.write("\n".join(map(str, a)) + "\n")
x = 10 ** 100  # 整数没有上限
p = 998244353
y = pow(3, x, p)  # 快速幂取模
z = pow(3, -1, p)  # 逆元，需 3.8 以上
g = math.gcd(12, 18); s = math.isqrt(10 ** 20)
c = math.comb(50, 25); l = math.lcm(4, 6)  # lcm 需 3.9 以上
q = -7 // 2  # 向下取整得 -4，和 C++ 不同，% 也是
f = Fraction(1, 3) + Fraction(1, 6)  # 精确分数
getcontext().prec = 50
d = Decimal(2).sqrt()  # 高精度小数
h = int("ff", 16); b = bin(10)[2:]; k = (10).bit_length()
pc = bin(10).count("1"); t = "abc"[::-1]; o = ord("a")
a.sort(key=lambda v: (-v, v % 3))  # 多关键字排序
a.sort(key=cmp_to_key(lambda u, v: u - v))  # 自定义比较
adj = [[] for _ in range(n)]  # 不要写 [[]] * n
dp = [[0] * m for _ in range(n)]
pq = [5, 1, 4]; heapify(pq); heappush(pq, 2)
mn = heappop(pq)  # 小根堆，大根堆存相反数
dq = deque([1, 2]); dq.appendleft(0); dq.pop()
i = bisect_left([1, 3, 5], 3)  # 同 lower_bound
cnt = Counter("abca"); dd = defaultdict(int)
pre = list(accumulate([1, 2, 3], initial=0))  # 前缀和
@lru_cache(maxsize=None)  # 记忆化搜索
def fib(v): return v if v < 2 else fib(v - 1) + fib(v - 2)
INF = float("inf")
# 字符串，不可修改，要改先转成 list(s)
s = "  Hello,World  "
w = s.strip().split(",")  # 去首尾空白，按逗号切
j = "-".join(w); u = s.upper(); lo = s.lower()
pos = s.find("o"); r = s.replace("l", "L")  # 找不到返回 -1
ok = "123".isdigit(); bg = s.strip().startswith("He")
cs = list("abc"); cs[0] = "z"; s2 = "".join(cs)
fs = f"{3.14159:.3f} {42:05d} {255:x}"  # 格式化输出
# 列表
a = [3, 1, 2]; a.append(5); a.insert(0, 9); a.pop()
a.extend([7, 7]); c7 = a.count(7); i1 = a.index(1)
b = sorted(a, reverse=True); a.reverse()
sq = [v * v for v in a if v % 2]  # 列表推导
mx = max(a, key=lambda v: -v); sm = sum(a)
for i, v in enumerate(a): pass
pairs = list(zip([1, 2], "ab")); rv = a[::-1]; sub = a[1:3]
# 集合 / 字典 / 元组
st = {1, 2}; st.add(3); st.discard(9); st2 = st | {4}
inter = st & {2, 3}; diff = st - {1}; has = 2 in st
d = {"x": 1}; d["y"] = 2; gv = d.get("z", 0)
for k, v in sorted(d.items(), key=lambda t: -t[1]): pass
dl = defaultdict(list); dl[1].append(2)
x1, y1 = (4, 5); x1, y1 = y1, x1  # 解包，交换
mc = Counter("aabbbc").most_common(2)  # 出现最多的两个
small = nsmallest(2, [5, 1, 4]); dq2 = deque([1, 2, 3])
dq2.rotate(1)  # 循环右移一位
```

数学 (2)

2.1 方程

$$ax^2+bx+c=0 \Rightarrow x=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

极值点为 $x=-b/2a$.

$$\begin{aligned}ax+by&=e\\cx+dy&=f\end{aligned}\Rightarrow\begin{aligned}x&=\frac{ed-bf}{ad-bc}\\y&=\frac{af-ec}{ad-bc}\end{aligned}$$

一般地, 对方程 $Ax=b$, 变量 x_i 的解为

$$x_i=\frac{\det A_i'}{\det A}$$

其中 A_i' 是把 A 的第 i 列换成 b 所得的矩阵.

2.2 递推

若 $a_n=c_1a_{n-1}+\cdots+c_ka_{n-k}$, 且 $r_1,\ldots,r_k$ 是 $x^k-c_1x^{k-1}-\cdots-c_k$ 的互异根, 则存在 $d_1,\ldots,d_k$ 使得

$$a_n=d_1r_1^n+\cdots+d_kr_k^n.$$

重根 r 会乘上多项式因子, 如 $a_n=(d_1n+d_2)r^n$.

2.3 三角函数

$$\sin(v+w)=\sin v\cos w+\cos v\sin w$$
$$\cos(v+w)=\cos v\cos w-\sin v\sin w$$

$$\tan(v+w)=\frac{\tan v+\tan w}{1-\tan v\tan w}$$
$$\sin v+\sin w=2\sin\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$\cos v+\cos w=2\cos\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$(V+W)\tan(v-w)/2=(V-W)\tan(v+w)/2$$

其中 V,W 是角 v,w 的对边长.

$$a\cos x+b\sin x=r\cos(x-\phi)$$
$$a\sin x+b\cos x=r\sin(x+\phi)$$

其中 $r=\sqrt{a^2+b^2},\phi=\operatorname{atan2}(b,a)$.

2.4 几何

2.4.1 三角形

边长: a,b,c

半周长: $p=\frac{a+b+c}{2}$

面积: $A=\sqrt{p(p-a)(p-b)(p-c)}$

外接圆半径: $R=\frac{abc}{4A}$

内切圆半径: $r=\frac{A}{p}$

中线长 (把三角形分成面积相等的两半): $m_a=\frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

角平分线长: $s_a = \sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$

正弦定理: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

余弦定理: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

正切定理: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

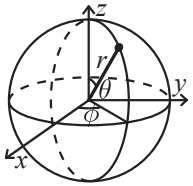
2.4.2 四边形

设边长 a, b, c, d, 对角线 e, f, 对角线夹角 θ, 面积 A, 记 $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

圆内接四边形对角之和为 $180°$, $ef = ac + bd$, 且 $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 球坐标



$$\begin{array}{ll} x = r\sin\theta\cos\phi & r = \sqrt{x^2+y^2+z^2} \\ y = r\sin\theta\sin\phi & \theta = \operatorname{acos}(z/\sqrt{x^2+y^2+z^2}) \\ z = r\cos\theta & \phi = \operatorname{atan2}(y, x) \end{array}$$

2.5 导数/积分

$$\begin{array}{ll} \frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx}\tan x = 1 + \tan^2 x & \frac{d}{dx}\arctan x = \frac{1}{1+x^2} \\ \int \tan ax = -\frac{\ln|\cos ax|}{a} & \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2} \\ \int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) & \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1) \end{array}$$

分部积分:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 求和

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 级数

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

2.8 概率论

设 X 是离散随机变量, 取值 x 的概率为 $p_X(x)$. 其期望 (均值) 为 $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$, 方差为 $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$, 其中 σ 是标准差. 若 X 是连续的, 则有概率密度函数 $f_X(x)$, 上面的求和改为积分, 并把 $p_X(x)$ 换成 $f_X(x)$.

期望是线性的:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

若 X 与 Y 独立,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.9 Markov 链

Markov 链 是一种离散随机过程, 下一状态只依赖当前状态. 设 $X_1, X_2, \ldots$ 是该过程产生的随机变量序列. 则有转移矩阵 $\mathbf{P} = (p_{ij})$, 其中 $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, 且 $\mathbf{p}^{(n)} = \mathbf{P}^n\mathbf{p}^{(0)}$ 是 X_n 的概率分布 (即 $p_i^{(n)} = \Pr(X_n = i)$), 其中 $\mathbf{p}^{(0)}$ 是初始分布.

平稳分布

若 $\pi = \pi\mathbf{P}$, 则称 π 为平稳分布. 若 Markov 链不可约 (任意状态都能到达任意状态), 则 $\pi_i = \frac{1}{\mathbb{E}(T_i)}$, 其中 $\mathbb{E}(T_i)$ 是相邻两次访问状态 i 的期望间隔. π_j/π_i 是相邻两次访问 i 之间访问状态 j 的期望次数.

在连通, 无向, 非二分的图上, 若等概率走向各邻居, 则 π_i 与点 i 的度数成正比.

遍历性

若极限分布与初始分布无关, 则称 Markov 链是遍历的. 有限 Markov 链遍历当且仅当它不可约且非周期 (即各环长的 gcd 为 1). $\lim_{k\to\infty}\mathbf{P}^k = \mathbf{1}\pi$.

吸收

若状态可分成两个集合 $\mathbf{A}$ 和 $\mathbf{G}$, 使 $\mathbf{A}$ 中的状态都是吸收态 ($p_{ii} = 1$), 且 $\mathbf{G}$ 中的状态都能到达 $\mathbf{A}$ 中的吸收态, 则称其为 A-链. 从 j 出发, 被状态 $i \in \mathbf{A}$ 吸收的概率为 $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik}p_{kj}$. 从 i 出发, 到被吸收的期望时间为 $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki}t_k$.

2.10 期望与概率技巧

- **期望线性性**: 不要求独立. 求个数的期望就拆成指示变量, $\mathbb{E}[\text{个数}] = \sum_i \Pr[\text{事件 } i]$.
- **尾和**: X 取非负整数时 $\mathbb{E}[X] = \sum_{k\geq 1}\Pr[X \geq k]$. 独立同分布的最大值: $\Pr[\max \leq k] = F(k)^n$.
- **期望 DP 倒推**: $E_i = c_i + \sum_j p_{ij}E_j$, 终点 $E = 0$, 按拓扑序从后往前算. 只有自环时移项: $E_i = (c_i + \sum_{j\neq i} p_{ij}E_j)/(1-p_{ii})$.
- **有环, 高斯消元**: 把 $E_i - \sum_j p_{ij}E_j = c_i$ 列成方程组, 用 `SolveLinear`, $O(n^3)$, n 约到 500. 取模时把除法换成乘逆元. 吸收概率同理.
- **只有一个未知量**: 设起点期望为 x, 其他状态都写成 $a_i + b_ix$ 一路推下去, 最后解 $x = a + bx$, 不用高斯消元.
- **二分答案**: 可以随时重来的题, 答案 x 出现在自己的转移里, 即 $E_i = \min(\text{继续}, x)$. 对猜的 x 做一遍 DP 得到 $f(x)$; 若 $f(x) < x$ 则真答案 $< x$, 否则 $\geq x$. 二分约 100 次.
- **方差**: DP 里同时维护 $\mathbb{E}[X]$ 和 $\mathbb{E}[X^2]$, 如 $\mathbb{E}[(X+1)^2] = \mathbb{E}[X^2] + 2\mathbb{E}[X] + 1$.
- **min-max 容斥**: $\mathbb{E}[\max S] = \sum_{\emptyset\neq T\subseteq S}(-1)^{|T|+1}\mathbb{E}[\min T]$. 用于全部出现的期望时间.
- **常用结论**: 成功率 p 的试验, 期望 $1/p$ 次成功一次. 集齐 n 种卡片: nH_n. n 个 $U[0,1]$ 中第 k 小的期望: $k/(n+1)$. 随机排列: 逆序对期望 $n(n-1)/4$, 不动点期望 1, 环数期望 H_n.
- **随机游走**: 树上从 u 走到父亲的期望步数是 $2 \cdot size(u) - 1$. 0 处反射, 从 0 走到 n 期望 n^2 步. 公平赌徒从 i 出发: 先到 n 的概率 i/n, 期望步数 $i(n-i)$.

- **精度**: 期望很大时用 `long double`; 方程太大时可以反复迭代 $E \leftarrow c + PE$ 直到收敛.

数据结构 (3)

OrderStatisticTree.h

Description: pb_ds 平衡树, 不可重集合, 不是 multiset. 支持求排名和第 k 小. 当 map 用: 把 null_type 换成值类型. 换元素类型只改 typedef 里的两个 int. 要可重: 元素换成 pair, first 存值, second 存互不相同的编号.
Usage: `ordered_set s; s.insert(x); s.erase(x);`
`int r = s.order_of_key(x); // 严格小于 x 的个数`
`int v = *s.find_by_order(k); // 第 k 小, k 从 0 开始`
`// k 不小于 s.size() 时返回 s.end()`
`s.lower_bound(x); s.size(); s.clear(); // 同 set`
Time: $\mathcal{O}(\log N)$

d3016d, 4 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;
typedef tree<int,null_type,less<int>,rb_tree_tag,
  tree_order_statistics_node_update>ordered_set;
```

HashMap.h

Description: pb_ds 哈希表, 用法同 unordered_map, 但快约 3 倍. 键 ll, 值 int, 要别的类型改 gp_hash_table 的前两个参数. chash 让高位也参与哈希; RND 为随机数, 防 CodeForces 上的 hack.
Usage: `h[key] = val; h.erase(key); h.clear();`
`if (h.find(key) != h.end()) ...; // 判断是否存在`
`for (auto [k, v] : h) ...; // 遍历, 顺序随机`
Time: 均摊 $\mathcal{O}(1)$

cb991d, 11 lines

```
#include <bits/extc++.h>
const unsigned long long RND=
  chrono::steady_clock::now().time_since_epoch().count();
struct chash{
  unsigned long long operator()(ll x)const{
    // C 为大奇数
    const unsigned long long C=(ll)(4e18*acos(0))|71;
    return __builtin_bswap64((x^RND)*C);
  }
};
__gnu_pbds::gp_hash_table<ll,int,chash>h({},{},{},{},{1<<16});
```

LazySegmentTree.h

Description: 懒标记线段树, 下标 a[1..n]. 默认: 区间加, 查区间和与区间最大值. 换用途只改 Node 的字段, operator+, build 的叶子, 标记数组, addtag 和 pushdown, 写法见下方懒标记写法. query 只合并非空的两半, 所以不需要单位元.
Usage: `n = ...; for (int i = 1; i <= n; i++) a[i] = ...;`
`build(1, 1, n); // 多组数据重新 build 即可, 会清空 lz`
`update(1, 1, n, l, r, v); // a[l..r] 都加 v`
`Node q = query(1, 1, n, l, r); // q.sum, q.mx`
Time: $\mathcal{O}(\log N)$

301f2a, 42 lines

```
const int N=2e5+5;
ll a[N];
struct Node{ll sum,mx;};
Node t[N<<2];
ll lz[N<<2]; // 整段待加的值
Node operator+(Node x,Node y){ // 合并左右儿子
  return {x.sum+y.sum,max(x.mx,y.mx)};
}
void pushup(int p){t[p]=t[p<<1]+t[p<<1|1];}
void addtag(int p,int l,int r,ll v){ // 整段 [l, r] 都加 v
  t[p].sum+=v*(r-l+1),t[p].mx+=v;
  lz[p]+=v;
}
void pushdown(int p,int l,int r){
  if(!lz[p])return;
  int m=(l+r)>>1;
  addtag(p<<1,l,m,lz[p]),addtag(p<<1|1,m+1,r,lz[p]);
  lz[p]=0;
}
void build(int p,int l,int r){
  lz[p]=0;
  if(l==r){t[p]={a[l],a[l]};return;}
  int m=(l+r)>>1;
  build(p<<1,l,m),build(p<<1|1,m+1,r);
  pushup(p);
}
void update(int p,int l,int r,int x,int y,ll v){ // 加 v
  if(x<=l&&r<=y){addtag(p,l,r,v);return;}
  pushdown(p,l,r);
  int m=(l+r)>>1;
  if(x<=m)update(p<<1,l,m,x,y,v);
  if(y>m)update(p<<1|1,m+1,r,x,y,v);
  pushup(p);
}
Node query(int p,int l,int r,int x,int y){ // [x, y]
  if(x<=l&&r<=y)return t[p];
  pushdown(p,l,r);
  int m=(l+r)>>1;
  if(y<=m)return query(p<<1,l,m,x,y);
  if(x>m)return query(p<<1|1,m+1,r,x,y);
  return query(p<<1,l,m,x,y)+query(p<<1|1,m+1,r,x,y);
}
```

懒标记写法

换用途只改: Node 的字段和合并两个儿子的 operator+, build 里叶子的初值, 标记数组, addtag 和 pushdown. addtag(p, l, r, v): 给整段 $[l,r]$ 打上标记 v, 要立刻更新 t[p], 并把 v 合并进 p 的旧标记, 旧标记先做, v 后做. update 命中整段时调用它, pushdown 给两个儿子时也调用它. 新加的标记数组都要在 build 里设成空标记. 区间加就是上面的模板.

- **区间赋值, 求和与最大值**: `update(1, 1, n, l, r, v)` 把 $a[l..r]$ 赋成 v.

```
// 标记: tg[p] 为 1 表示整段待赋成 lz[p]
ll lz[N<<2];bool tg[N<<2]; // build 里清 0
void addtag(int p,int l,int r,ll v){ // 整段赋成 v
  t[p].sum=v*(r-l+1),t[p].mx=v;
  lz[p]=v,tg[p]=1; // 新赋值直接覆盖旧标记
}
void pushdown(int p,int l,int r){
  if(!tg[p])return;
  int m=(l+r)>>1;
  addtag(p<<1,l,m,lz[p]),addtag(p<<1|1,m+1,r,lz[p]);
  tg[p]=0;
}
```

- **赋值和加混用**: update 多一个参数 op, 为 1 赋值, 为 0 加.

```
// 标记: 若 tg[p] 先整段赋成 sv[p], 再整段加 ad[p]
ll sv[N<<2],ad[N<<2];bool tg[N<<2]; // build 里清 0
void setv(int p,int l,int r,ll v){ // 整段赋成 v
  t[p].sum=v*(r-l+1),t[p].mx=v;
  sv[p]=v,tg[p]=1,ad[p]=0;
}
void addv(int p,int l,int r,ll v){ // 整段加 v
  t[p].sum+=v*(r-l+1),t[p].mx+=v;
  ad[p]+=v;
}
void pushdown(int p,int l,int r){
  int m=(l+r)>>1;
  if(tg[p]){
    setv(p<<1,l,m,sv[p]),setv(p<<1|1,m+1,r,sv[p]);
    tg[p]=0;
  }
  if(ad[p]){
    addv(p<<1,l,m,ad[p]),addv(p<<1|1,m+1,r,ad[p]);
    ad[p]=0;
  }
}
void update(int p,int l,int r,int x,int y,int op,ll v){
  if(x<=l&&r<=y){
    if(op)setv(p,l,r,v);
    else addv(p,l,r,v);
    return;
  } // 其余同模板, 递归时也传 op
```

- **区间乘再加, 求和取模**: `update(1, 1, n, l, r, k, b)` 把 $a[l..r]$ 每个数 v 变成 $kv+b$, 要求 $0 \le k, b < M$.

```
const ll M=998244353;
struct Node{ll sum;};
Node operator+(Node x,Node y){return {(x.sum+y.sum)%M};}
// 标记: 整段每个数先乘 mu[p] 再加 ad[p]
ll mu[N<<2],ad[N<<2]; // build 里 mu[p]=1,ad[p]=0
// 叶子 t[p]={a[l]%M}, 不要 mx
void addtag(int p,int l,int r,ll k,ll b){
  t[p].sum=(k*t[p].sum+b*(r-l+1))%M;
  mu[p]=mu[p]*k%M,ad[p]=(ad[p]*k+b)%M;
}
void pushdown(int p,int l,int r){
  if(mu[p]==1&&!ad[p])return;
  int m=(l+r)>>1;
  addtag(p<<1,l,m,mu[p],ad[p]);
  addtag(p<<1|1,m+1,r,mu[p],ad[p]);
  mu[p]=1,ad[p]=0;
}
// update 的参数 v 换成 k, b, 命中整段时 addtag(p,l,r,k,b)
```

- **01 序列区间取反, 数 1 的个数**: `update(1, 1, n, l, r)`, $a[i]$ 为 0 或 1.

```
struct Node{int cnt;}; // 区间里 1 的个数
Node operator+(Node x,Node y){return {x.cnt+y.cnt};}
bool rev[N<<2]; // 取反标记, build 里清 0
// 叶子 t[p]={(int)a[l]}
void addtag(int p,int l,int r){ // 整段 0 和 1 互换
  t[p].cnt=(r-l+1)-t[p].cnt,rev[p]^=1;
}
void pushdown(int p,int l,int r){
  if(!rev[p])return;
  int m=(l+r)>>1;
  addtag(p<<1,l,m),addtag(p<<1|1,m+1,r);
  rev[p]=0;
}
// update 去掉参数 v, 命中整段时 addtag(p,l,r)
```

- **最大子段和**: 只有单点赋值 `modify(1, 1, n, x, v)`, 不需要标记. `query(1, 1, n, l, r).best` 为 $a[l..r]$ 的最大非空子段和.

```
struct Node{ll sum,pre,suf,best;}; // 前缀, 后缀, 答案
Node operator+(Node x,Node y){
  return {x.sum+y.sum,max(x.pre,x.sum+y.pre),
          max(y.suf,y.sum+x.suf),
          max({x.best,y.best,x.suf+y.pre})};
}
// 叶子 t[p]={a[l],a[l],a[l],a[l]}
```

```
  // 删去 lz, addtag, pushdown 及其所有调用
  void modify(int p,int l,int r,int x,ll v){ // a[x] 赋成 v
    if(l==r){t[p]={v,v,v,v};return;}
    int m=(l+r)>>1;
    if(x<=m)modify(p<<1,l,m,x,v);
    else modify(p<<1|1,m+1,r,x,v);
    pushup(p);
  }
```

PersistentSegmentTree.h

Description: 可持久化线段树, 即主席树, 下标 [1, n]. 每次单点加产生一个新版本, 只新建 $O(\log n)$ 个节点, 旧版本不变. 节点 0 是空树, rt[0] = 0. 区间第 k 小: 把值离散化到 1..m, 版本 i 在 a[i] 处加 1, 则 rt[r] 减 rt[l - 1] 就是 a[l..r] 中各值的出现次数.
Usage: `rt[i] = update(rt[i - 1], 1, n, x, v); // 新版本: 第 x 位加 v`
`query(rt[i], 1, n, l, r); // 版本 i 中 [l, r] 的和`
`// 区间第 k 小: b 为排序去重后的值, m = b 的大小`
`rt[i] = update(rt[i - 1], 1, m, id(a[i]), 1); // id 从 1 开始`
`b[kth(rt[l - 1], rt[r], 1, m, k)]; // k 从 1 开始`
多组数据: `tot = 0`.
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(\log N)$ 每次操作

1ecd57, 26 lines

```
const int N=2e5+5,MX=N*20; // MX: 节点数, 约 N * (log n + 2)
int ls[MX],rs[MX],tot,rt[N];
ll sum[MX];
int update(int pre,int l,int r,int x,ll v){ // 返回新根
  int p=++tot;
  ls[p]=ls[pre],rs[p]=rs[pre],sum[p]=sum[pre]+v;
  if(l==r)return p;
  int m=(l+r)>>1;
  if(x<=m)ls[p]=update(ls[pre],l,m,x,v);
  else rs[p]=update(rs[pre],m+1,r,x,v);
  return p;
}
ll query(int p,int l,int r,int x,int y){ // [x, y] 的和
  if(!p||y<l||r<x)return 0;
  if(x<=l&&r<=y)return sum[p];
  int m=(l+r)>>1;
  return query(ls[p],l,m,x,y)+query(rs[p],m+1,r,x,y);
}
// 版本 v 减版本 u 中第 k 小的位置 (k 从 1 开始)
int kth(int u,int v,int l,int r,ll k){
  if(l==r)return l;
  int m=(l+r)>>1;
  ll c=sum[ls[v]]-sum[ls[u]];
  if(k<=c)return kth(ls[u],ls[v],l,m,k);
  return kth(rs[u],rs[v],m+1,r,k-c);
}
```

UnionFindRollback.h

Description: 并查集, 点 1..n. 上半: 普通并查集, 路径压缩. 下半: 可撤销并查集, 按大小合并, 不压缩路径. 每次成功合并压栈一次, rollback(t) 撤销到 time() 等于 t 时的状态. 两者互相独立, 只抄需要的.
Usage: `init(n); unite(x, y); find(x) == find(y); // 普通`
`initR(n); uniteR(x, y); findR(x); siz[findR(x)]; // 可撤销`
`int t = time(); uniteR(x, y); ...; rollback(t);`
Time: 普通 均摊 $\mathcal{O}(\log N)$, 实际接近 $\mathcal{O}(1)$; 可撤销 $\mathcal{O}(\log N)$

6d9aa1, 33 lines

```
const int N=2e5+5;
int fa[N];
void init(int n){for(int i=1;i<=n;i++)fa[i]=i;}
int find(int x){ // 路径压缩: 减半写法, 不递归, 不怕爆栈
  while(fa[x]!=x)x=fa[x]=fa[fa[x]];
  return x;
}
bool unite(int x,int y){ // 已在同一集合返回 0
  x=find(x),y=find(y);
  if(x==y)return 0;
  fa[x]=y;return 1;
}

int pa[N],siz[N],stk[N],top; // stk: 被挂到别人下面的根
void initR(int n){
  top=0;
  for(int i=1;i<=n;i++)pa[i]=i,siz[i]=1;
}
int findR(int x){while(pa[x]!=x)x=pa[x];return x;}
bool uniteR(int x,int y){
  x=findR(x),y=findR(y);
  if(x==y)return 0;
  if(siz[x]>siz[y])swap(x,y); // 小的挂到大的下面
  pa[x]=y,siz[y]+=siz[x],stk[++top]=x;
  return 1;
}
int time(){return top;}
void rollback(int t){ // 撤销到 time() 等于 t 时
  while(top>t){
    int x=stk[top--];
    siz[pa[x]]-=siz[x],pa[x]=x;
  }
}
```

LiChaoTree.h

Description: 李超树, 整数 x 取值于 [lo, hi), 动态开点, 不用离散化. addLine 插入直线 $y = kx + b$, addSeg 插入只在 $x \in [x_1, x_2)$ 上存在的线段. query(x) 返回 x 处的最大值, 没有线段覆盖 x 时返回 LLONG_MIN. 求最小值: 插入 $(-k, -b)$, 再把答案取负. 要求 $|kx + b| < 9 \cdot 10^{18}$, $hi - lo < 9 \cdot 10^{18}$.
Usage: `init(-1e9, 1e9 + 1); // 每组数据调用一次`
`addLine({k, b}); addSeg({k, b}, x1, x2); ll y = query(x);`
Time: 直线和查询 $\mathcal{O}(\log C)$, 线段 $\mathcal{O}\left(\log^2 C\right)$, $C = hi - lo$

b11c69, 36 lines

```
// 节点数: 每条直线最多 1 个, 每条线段最多约 2 log C 个
const int M=4e6+5;
struct Line{ll k,b;};
Line ln[M];
int ls[M],rs[M],tot,root;bool has[M]; // has: 节点上有线
ll lo,hi;
ll calc(Line a,ll x){return a.k*x+a.b;}
void init(ll L,ll R){lo=L,hi=R,tot=root=0;}
// 节点 p 管 [l, r), 在 [x, y) 上插入 a
void ins(int&p,ll l,ll r,ll x,ll y,Line a){
  if(y<=l||r<=x)return;
  if(!p)p=++tot,ls[p]=rs[p]=0,has[p]=0;
  ll m=l+(r-l)/2;
  if(x<=l&&r<=y){
    if(!has[p]){ln[p]=a,has[p]=1;return;}
    if(calc(a,m)>calc(ln[p],m))swap(a,ln[p]);
    // 现在 a 在 m 处不优, 只可能在一侧更优
    if(r-l==1||a.k==ln[p].k)return;
    if(a.k<ln[p].k)ins(ls[p],l,m,x,y,a);
    else ins(rs[p],m,r,x,y,a);
    return;
  }
  ins(ls[p],l,m,x,y,a),ins(rs[p],m,r,x,y,a);
}
void addLine(Line a){ins(root,lo,hi,lo,hi,a);}
void addSeg(Line a,ll x1,ll x2){ins(root,lo,hi,x1,x2,a);}
ll query(ll x){
  ll res=LLONG_MIN,l=lo,r=hi;
  for(int p=root;p;){
    if(has[p])res=max(res,calc(ln[p],x));
    ll m=l+(r-l)/2;
    if(x<m)p=ls[p],r=m;
    else p=rs[p],l=m;
  }
  return res;
}
```

FHQTreap.h

Description: FHQ treap (split/merge, 无旋转), 节点 0 表示空. 两种用法: (1) 按值有序的可重集合: insert, erase, getrank, kth, pre, nxt; (2) 按位置的序列: 按大小 split, 区间翻转, 区间和. 两种用法不要混在同一棵树上. sum 和 rev 只给序列用. kth, pre, nxt 要保证答案存在. N 为插入总次数, 删掉的节点不回收.
Usage: `tot = root = 0; // 多组数据清空`
`insert(v); erase(v); getrank(v); kth(k); pre(v); nxt(v);`
`// 建序列: 依次加到末尾`
`for (int i = 1; i <= n; i++)`
`root = merge(root, newnode(a[i]));`
`reverse(l, r); rangeSum(l, r); // 位置从 1 开始`
`vector<ll> o; dfs(root, o); // 中序遍历得到当前序列`
Time: 每次操作期望 $\mathcal{O}(\log N)$

3ff5c7, 92 lines

```
const int N=2e5+5; // 插入总次数
int ls[N],rs[N],siz[N],tot,root;
unsigned rnd[N];ll val[N],sum[N];bool rev[N];
mt19937 rng(12345);
int newnode(ll v){
  int p=++tot;
  ls[p]=rs[p]=0,siz[p]=1,rev[p]=0;
  rnd[p]=rng(),val[p]=sum[p]=v;
  return p;
}
void pushup(int p){
  siz[p]=siz[ls[p]]+siz[rs[p]]+1;
  sum[p]=sum[ls[p]]+sum[rs[p]]+val[p];
}
void pushdown(int p){ // 下传翻转标记
  if(!rev[p])return;
  swap(ls[p],rs[p]);
  if(ls[p])rev[ls[p]]^=1;
  if(rs[p])rev[rs[p]]^=1;
  rev[p]=0;
}
void splitv(int p,ll v,int&x,int&y){ // 值不超过 v 的给 x
  if(!p){x=y=0;return;}
  pushdown(p);
  if(val[p]<=v)x=p,splitv(rs[p],v,rs[p],y);
  else y=p,splitv(ls[p],v,x,ls[p]);
  pushup(p);
}
void split(int p,int k,int&x,int&y){ // 前 k 个给 x
  if(!p){x=y=0;return;}
  pushdown(p);
  if(siz[ls[p]]<k)x=p,split(rs[p],k-siz[ls[p]]-1,rs[p],y);
  else y=p,split(ls[p],k,x,ls[p]);
  pushup(p);
}
int merge(int x,int y){ // x 的元素都在 y 前面
  if(!x||!y)return x|y;
  if(rnd[x]<rnd[y]){
    pushdown(x),rs[x]=merge(rs[x],y),pushup(x);
    return x;
  }
  pushdown(y),ls[y]=merge(x,ls[y]),pushup(y);
  return y;
}
// ——— 用法 1: 按值的可重集合 ———
void insert(ll v){
  int x,y;
  splitv(root,v,x,y);
  root=merge(merge(x,newnode(v)),y);
}
void erase(ll v){ // 删一个 v, 没有则不变
  int x,y,z;
  splitv(root,v,x,z),splitv(x,v-1,x,y);
```

```
  y=merge(ls[y],rs[y]);
  root=merge(merge(x,y),z);
}
int getrank(ll v){ // 严格小于 v 的个数 + 1
  int x,y;
  splitv(root,v-1,x,y);
  int r=siz[x]+1;
  root=merge(x,y);
  return r;
}
ll kth(int k){ // 第 k 小, k 从 1 开始
  int p=root;
  while(1){
    if(k<=siz[ls[p]])p=ls[p];
    else if(k==siz[ls[p]]+1)return val[p];
    else k-=siz[ls[p]]+1,p=rs[p];
  }
}
ll pre(ll v){return kth(getrank(v)-1);} // 小于 v 的最大值
ll nxt(ll v){return kth(getrank(v+1));} // 大于 v 的最小值
// ——— 用法 2: 序列, 位置从 1 开始 ———
void reverse(int l,int r){
  int x,y,z;
  split(root,r,x,z),split(x,l-1,x,y);
  rev[y]^=1;
  root=merge(merge(x,y),z);
}
ll rangeSum(int l,int r){
  int x,y,z;
  split(root,r,x,z),split(x,l-1,x,y);
  ll s=sum[y];
  root=merge(merge(x,y),z);
  return s;
}
void dfs(int p,vector<ll>&o){ // 中序遍历
  if(!p)return;
  pushdown(p),dfs(ls[p],o);
  o.push_back(val[p]),dfs(rs[p],o);
}
```

FenwickTree.h

Description: 树状数组, 下标 1..n. 上半: 单点加, 前缀和, 二分找前缀和. 下半: 区间加, 区间和, 用两个树状数组维护差分 d[i] 和 i * d[i]. 两半互相独立, 只抄需要的.
Usage: `n = ...; // 多组数据: 把 c 或 c1, c2 的 1..n 清 0`
`add(x, v); // a[x] += v`
`ask(x); // a[1] + ... + a[x]`
`ask(r) - ask(l - 1); // a[l..r] 的和`
`search(s); // 最小的 x 使 a[1..x] 的和不小于 s, 要求 a 非负`
`rangeAdd(l, r, v); rangeSum(l, r); // 区间加, 区间和`
Time: $\mathcal{O}(\log N)$

4d686e, 27 lines

```
const int N=2e5+5; // 要求 n < 2^21
int n;
ll c[N];
void add(int x,ll v){for(;x<=n;x+=x&-x)c[x]+=v;}
ll ask(int x){
  ll r=0;
  for(;x;x-=x&-x)r+=c[x];
  return r;
}
int search(ll s){ // 不存在返回 n + 1
  int x=0;
  for(int i=1<<20;i;i>>=1)
    if(x+i<=n&&c[x+i]<s)x+=i,s-=c[x];
  return x+1;
}

ll c1[N],c2[N]; // 差分 d[i] 和 i * d[i]
void radd(int x,ll v){
  for(int i=x;i<=n;i+=i&-i)c1[i]+=v,c2[i]+=v*x;
}
void rangeAdd(int l,int r,ll v){radd(l,v),radd(r+1,-v);}
ll rsum(int x){ // a[1..x] 的和
  ll s=0;
  for(int i=x;i;i-=i&-i)s+=(x+1)*c1[i]-c2[i];
  return s;
}
ll rangeSum(int l,int r){return rsum(r)-rsum(l-1);}
```

FenwickTree2d.h

Description: 二维树状数组, 矩阵 a[1..n][1..m]: 单点加, 子矩阵求和. 独立使用, 变量名和一维的 FenwickTree.h 相同, 不要同时抄. 坐标很大但修改点事先已知时, 改用离线: 离散化 + 一维树状数组扫描线.
Usage: `n = ..., m = ...; // 多组数据: 把 c[1..n][1..m] 清 0`
`add(x, y, v); // a[x][y] += v`
`ask(x, y); // 子矩阵 [1..x][1..y] 的和`
`query(x1, y1, x2, y2); // 子矩阵 [x1..x2][y1..y2] 的和`
Time: $\mathcal{O}(\log N \log M)$

358993, 16 lines

```
const int N=1005,M=1005;
int n,m;
ll c[N][M];
void add(int x,int y,ll v){
  for(int i=x;i<=n;i+=i&-i)
    for(int j=y;j<=m;j+=j&-j)c[i][j]+=v;
}
ll ask(int x,int y){
  ll r=0;
  for(int i=x;i;i-=i&-i)
    for(int j=y;j;j-=j&-j)r+=c[i][j];
  return r;
}
ll query(int x1,int y1,int x2,int y2){
  return ask(x2,y2)-ask(x1-1,y2)-ask(x2,y1-1)+ask(x1-1,y1-1);
}
```

MoQueries.h

Description: 普通莫队, 离线回答 q 个区间询问 a[l..r]. 只需写 add(i), del(i): 把 a[i] 加入或移出当前区间并维护答案 cur. 下面的 add, del 是例子: 区间不同值个数, 值已离散化到 1..n, 按题意改. 块长 $B = n/\sqrt{q}$, 奇偶块按 r 交替排序.
Usage: `读入 n, q, a[1..n]; 第 i 个询问 qs[i] = {l, r, i};`
`solve(); // 之后 ans[i] 为第 i 个询问的答案`
`多组数据: 清空 cnt 和 cur`
Time: $\mathcal{O}\left(n\sqrt{q}\right)$ 次 add/del

b262e8, 23 lines

```
const int N=2e5+5;
int n,q,B,a[N];
struct Query{int l,r,id;}qs[N];
ll ans[N],cur;
int cnt[N];
void add(int i){if(cnt[a[i]]++==0)cur++;} // a[i] 进入
void del(int i){if(--cnt[a[i]]==0)cur--;} // a[i] 离开
bool cmp(const Query&x,const Query&y){
  if(x.l/B!=y.l/B)return x.l<y.l;
  return (x.l/B)&1?x.r<y.r:x.r>y.r;
}
void solve(){
  B=max(1,(int)(n/sqrt(q)));
  sort(qs+1,qs+q+1,cmp);
  int L=1,R=0; // 当前区间 [L, R), 开始为空
  for(int i=1;i<=q;i++){
    while(L>qs[i].l)add(--L);
    while(R<qs[i].r)add(++R);
    while(L<qs[i].l)del(L++);
    while(R>qs[i].r)del(R--);
    ans[qs[i].id]=cur;
  }
}
```

数值计算 (4)

4.1 多项式与线性递推

Polynomial.h

Description: 实系数多项式, 下标从 0 开始, $a[i]$ 为 x^i 的系数. peval 用秦九韶 (Horner) 法求值; pderiv 原地求导; pdivroot 原地除以 $(x - x_0)$, 余数丢弃 (x_0 应为根).
Usage: `vector<double> a={2,-3,1}; // x^2-3x+2`
`double y=peval(a,0.5); // 0.75`
`pdivroot(a,1); // a 变为 {-2,1}, 即 x-2`
`pderiv(a); // a 变为 {1}`
Time: 均为 $\mathcal{O}(n)$

ee59e5, 16 lines

```
double peval(const vector<double>&a,double x){
  double res=0;
  for(int i=(int)a.size()-1;i>=0;i--)res=res*x+a[i];
  return res;
}
void pderiv(vector<double>&a){ // 求导, 次数减 1
  for(int i=1;i<(int)a.size();i++)a[i-1]=i*a[i];
  a.pop_back();
}
void pdivroot(vector<double>&a,double x0){ // 综合除法
  double b=a.back(),c;
  a.back()=0;
  for(int i=(int)a.size()-2;i>=0;i--)
    c=a[i],a[i]=a[i+1]*x0+b,b=c;
  a.pop_back();
}
```

PolyInterpolate.h

Description: 给定 n 个点 (x_i, y_i) (x_i 互不相同), 求经过它们的 $n-1$ 次多项式 $p(x) = a_0 + a_1 x + \cdots + a_{n-1}x^{n-1}$ (牛顿插值). 下标从 0 开始. 为保证精度, 可取 $x_k = c\cos(k\pi/(n-1))$.
Usage: `vector<double> x={0,1,2}, y={1,2,5};`
`vector<double> a=interpolate(x,y); // a={1,0,1}`
Time: $\mathcal{O}\left(n^2\right)$

abe767, 17 lines

```
vector<double>interpolate(vector<double>x,vector<double>y){
  int n=x.size();
  vector<double>res(n),tmp(n);
  // 均差: 结束后 y[k] 为 f[x_0..x_k]
  for(int k=0;k<n-1;k++)
    for(int i=k+1;i<n;i++)y[i]=(y[i]-y[k])/(x[i]-x[k]);
  // tmp 为 (x-x_0)...(x-x_{k-1}) 的系数
  double last=0;
  tmp[0]=1;
  for(int k=0;k<n;k++)
    for(int i=0;i<n;i++){
      res[i]+=y[k]*tmp[i];
      swap(last,tmp[i]);
      tmp[i]-=last*x[k];
    }
  return res;
}
```

BerlekampMassey.h

Description: Berlekamp-Massey: 由数列前 $2n$ 项求出最短的线性递推式 (模素数 MOD). 返回 C (下标从 0 开始), 满足 $s_i = \sum_j C_j s_{i-j-1}$. 常用于暴力算出前若干项后猜递推式, 再配合 LinearRecurrence 求第 k 项. 输入须在 $[0, MOD)$ 内.
Usage: `vector<ll> C=BM({0,1,1,3,5,11}); // C={1,2}`
`// 即 s[i]=s[i-1]+2*s[i-2]`
Time: $\mathcal{O}\left(n^2\right)$

b782ae, 30 lines

```
const ll MOD=998244353; // 素数模, 可改
ll qpow(ll a,ll b){
```

```
  ll r=1;
  for(a%=MOD;b;b>>=1,a=a*a%MOD)
    if(b&1)r=r*a%MOD;
  return r;
}
vector<ll>BM(vector<ll>s){
  int n=s.size(),L=0,m=0;
  if(n==0)return {};
  vector<ll>C(n),B(n),T;
  C[0]=B[0]=1;
  ll b=1;
  for(int i=0;i<n;i++){
    m++;
    ll d=s[i]%MOD; // 当前递推式的偏差
    for(int j=1;j<=L;j++)d=(d+C[j]*s[i-j])%MOD;
    if(!d)continue;
    T=C;
    ll coef=d*qpow(b,MOD-2)%MOD;
    for(int j=m;j<n;j++)
      C[j]=(C[j]+MOD-coef*B[j-m]%MOD)%MOD;
    if(2*L>i)continue;
    L=i+1-L;B=T;b=d;m=0;
  }
  C.resize(L+1);
  C.erase(C.begin());
  for(auto&x:C)x=(MOD-x)%MOD;
  return C;
}
```

LinearRecurrence.h

Description: 已知 n 阶线性递推 $S_i=\sum_{j=0}^{n-1} tr_j S_{i-j-1}$ 及前 n 项 $S_0..S_{n-1}$, 求第 k 项 (k 从 0 开始, 模 MOD). 用多项式取模快速幂, 比矩阵快速幂快. MOD 来自 BerlekampMassey.h (可直接改成自己的常量).
Usage: `ll f=linearRec({0,1},{1,1},k); // 第 k 个 Fibonacci`
`vector<ll> tr=BM(s); // 先猜递推式`
`ll v=linearRec(s,tr,k); // s 至少有 tr.size() 项`
Time: $\mathcal{O}\left(n^2 \log k\right)$

"BerlekampMassey.h" aa5b9e, 27 lines

```
// 多项式 a*b 对特征多项式取模, a, b 长 n+1
vector<ll>combine(const vector<ll>&a,const vector<ll>&b,
    const vector<ll>&tr){
  int n=tr.size();
  vector<ll>res(2*n+1);
  for(int i=0;i<=n;i++)
    for(int j=0;j<=n;j++)
      res[i+j]=(res[i+j]+a[i]*b[j])%MOD;
  for(int i=2*n;i>n;i--)
    for(int j=0;j<n;j++)
      res[i-1-j]=(res[i-1-j]+res[i]*tr[j])%MOD;
  res.resize(n+1);
  return res;
}
ll linearRec(vector<ll>S,vector<ll>tr,ll k){
  int n=tr.size();
  if(n==0)return 0; // 空递推: 全为 0
  vector<ll>pol(n+1),e(n+1);
  pol[0]=e[1]=1;
  for(k++;k;k>>=1){
    if(k&1)pol=combine(pol,e,tr);
    e=combine(e,e,tr);
  }
  ll res=0;
  for(int i=0;i<n;i++)res=(res+pol[i+1]*S[i])%MOD;
  return res;
}
```

4.2 最优化与积分

GoldenSectionSearch.h

Description: 黄金分割搜索: 在 $[a,b]$ 上求单峰函数 f 的最小值点 (只有一个局部极小值, 没有局部极大值), 误差约 eps. 求最大值把 $f_1<f_2$ 改成 $f_1>f_2$. 每次只算一次 f, 比三分快. 离散版本见 Various 章 TernarySearch.h.
Usage: `double f(double x){return 4+x+.3*x*x;} // 自己写 f`
`double xm=gss(-1000,1000); // xm 约为 -5/3`
Time: $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$

077039, 16 lines

```
double f(double x); // 要最小化的函数, 在别处自己实现
double gss(double a,double b){
  // r 要足够精确, 才能保证 a < x1 < x2 < b
  double r=(sqrt(5.0)-1)/2,eps=1e-7;
  double x1=b-r*(b-a),x2=a+r*(b-a);
  double f1=f(x1),f2=f(x2);
  while(b-a>eps)
    if(f1<f2){ // 求最大值改为 f1 > f2
      b=x2;x2=x1;f2=f1;
      x1=b-r*(b-a);f1=f(x1);
    }else{
      a=x1;x1=x2;f1=f2;
      x2=a+r*(b-a);f2=f(x2);
    }
  return a;
}
```

IntegrateAdaptive.h

Description: 自适应 Simpson 积分, 求 $\int_l^r f(x)\,dx$. 被积函数为全局函数 `double f(double x)`, 自己实现. 多重积分: 让 f 里再调用另一个 被积函数的积分 (复制一份改名即可).
Usage: `double f(double x){return sin(x);} // 自己写 f`
`double s=integrate(0,acos(-1)); // s 约为 2`
`double t=integrate(0,1,1e-10); // 第三个参数为精度`
Time: 与 f 的光滑程度有关

b647bf, 15 lines

```
double f(double x); // 被积函数, 在别处自己实现
double simpson(double l,double r){
  double m=(l+r)/2;
  return (f(l)+4*f(m)+f(r))*(r-l)/6;
}
// dep: 至少再细分的层数, 防止一开始就误判收敛
double asr(double l,double r,double eps,double S,int dep){
  double m=(l+r)/2,L=simpson(l,m),R=simpson(m,r);
  if(dep<=0&&(fabs(L+R-S)<=15*eps||r-l<1e-10))
    return L+R+(L+R-S)/15;
  return asr(l,m,eps/2,L,dep-1)+asr(m,r,eps/2,R,dep-1);
}
double integrate(double l,double r,double eps=1e-8){
  return asr(l,r,eps,simpson(l,r),6);
}
```

Simplex.h

Description: 单纯形法解线性规划: 在 $Ax\le b,\ x\ge 0$ 下最大化 c^Tx. 无解返回 -INF, 无界返回 INF, 否则返回最大值, 最优解存在 x 中. 下标从 1 开始: $A[1..m][1..n]$, $b[1..m]$, $c[1..n]$, $x[1..n]$. 求最小化把 c 取负; 约束 $\ge$ 两边乘 -1; 等式拆成两个不等式. 不保证数值稳定. $x=0$ 可行 (即 $b\ge 0$) 时更快.
Usage: `m=3,n=2; // 填好 A, b, c, 多组数据重新填即可`
`double v=simplex(m,n); // v 为最大值, 最优解在 x[1..n]`
`if(v==-INF) ... // 无解; v==INF 为无界`
Time: $\mathcal{O}\left(nm\cdot \#pivots\right)$, 最坏指数级, 实际很快

1781aa, 68 lines

```
const int MM=205,NN=205; // 约束个数, 变量个数上限
const double eps=1e-8,INF=1e18;
double A[MM][NN],b[MM],c[NN],x[NN]; // 输入输出
int m,n,bv[MM],nv[NN]; // 基变量, 非基变量
double D[MM][NN]; // 单纯形表, 下标从 0 开始
bool lessp(double a,int i,double b,int j){
  return make_pair(a,i)<make_pair(b,j);
}
void pivot(int r,int s){
  double inv=1/D[r][s];
  for(int i=0;i<m+2;i++)
    if(i!=r&&fabs(D[i][s])>eps){
      double t=D[i][s]*inv;
      for(int j=0;j<n+2;j++)D[i][j]-=D[r][j]*t;
      D[i][s]=D[r][s]*t;
    }
  for(int j=0;j<n+2;j++)if(j!=s)D[r][j]*=inv;
  for(int i=0;i<m+2;i++)if(i!=r)D[i][s]*=-inv;
  D[r][s]=inv;
  swap(bv[r],nv[s]);
}
bool run(int phase){
  int x=m+phase-1;
  while(1){
    int s=-1; // 入基列 (Bland 规则防循环)
    for(int j=0;j<=n;j++)
      if(nv[j]!=-phase&&(s==-1||
          lessp(D[x][j],nv[j],D[x][s],nv[s])))s=j;
    if(D[x][s]>=-eps)return true;
    int r=-1; // 出基行
    for(int i=0;i<m;i++){
      if(D[i][s]<=eps)continue;
      if(r==-1||lessp(D[i][n+1]/D[i][s],bv[i],
          D[r][n+1]/D[r][s],bv[r]))r=i;
    }
    if(r==-1)return false;
    pivot(r,s);
  }
}
double simplex(int _m,int _n){
  m=_m;n=_n;
  for(int i=0;i<m+2;i++)
    for(int j=0;j<n+2;j++)D[i][j]=0;
  for(int i=0;i<m;i++){
    for(int j=0;j<n;j++)D[i][j]=A[i+1][j+1];
    bv[i]=n+i;D[i][n]=-1;D[i][n+1]=b[i+1];
  }
  for(int j=0;j<n;j++)nv[j]=j,D[m][j]=-c[j+1];
  nv[n]=-1;D[m+1][n]=1;
  int r=0;
  for(int i=1;i<m;i++)if(D[i][n+1]<D[r][n+1])r=i;
  if(D[r][n+1]<-eps){ // x=0 不可行, 先找可行解
    pivot(r,n);
    if(!run(2)||D[m+1][n+1]<-eps)return -INF;
    for(int i=0;i<m;i++)
      if(bv[i]==-1){
        int s=0;
        for(int j=1;j<=n;j++)
          if(lessp(D[i][j],nv[j],D[i][s],nv[s]))s=j;
        pivot(i,s);
      }
  }
  bool ok=run(1);
  for(int j=1;j<=n;j++)x[j]=0;
  for(int i=0;i<m;i++)
    if(bv[i]<n)x[bv[i]+1]=D[i][n+1];
  return ok?D[m][n+1]:INF;
}
```

4.3 矩阵

Determinant.h

Description: 高斯消元 (列主元) 求实矩阵的行列式. 下标从 1 开始, $a[1..n][1..n]$ 会被破坏. 整数矩阵取模请用 IntDeterminant.
Usage: `// 填好 a[1..n][1..n], 多组数据重新填即可`
`double d=det(n);`
Time: $\mathcal{O}\left(n^3\right)$

cb505b, 19 lines

```
const int N=505; // 矩阵大小上限
double a[N][N];
double det(int n){
  double res=1;
  for(int i=1;i<=n;i++){
    int p=i; // 选绝对值最大的主元
    for(int j=i+1;j<=n;j++)
      if(fabs(a[j][i])>fabs(a[p][i]))p=j;
    if(a[p][i]==0)return 0;
    if(p!=i)swap(a[p],a[i]),res=-res;
    res*=a[i][i];
    for(int j=i+1;j<=n;j++){
      double t=a[j][i]/a[i][i];
      if(t!=0)
        for(int k=i;k<=n;k++)a[j][k]-=t*a[i][k];
    }
  }
  return res;
}
```

IntDeterminant.h

Description: 求整数矩阵行列式模 p. 用辗转相除消元, 不需要求逆元, 所以 p 不必是素数 (如矩阵树定理模合数). 要求 p^2 不超过 long long. 下标从 1 开始, $a[1..n][1..n]$ 会被破坏. 可以有负数.
Usage: `// 填好 a[1..n][1..n], 多组数据重新填即可`
`ll d=detMod(n,p); // 结果在 [0,p) 内`
Time: $\mathcal{O}\left(n^2(n+\log p)\right)$

d47883, 20 lines

```
const int N=505; // 矩阵大小上限
ll a[N][N];
ll detMod(int n,ll p){
  ll res=1;
  for(int i=1;i<=n;i++)
    for(int j=1;j<=n;j++)a[i][j]=(a[i][j]%p+p)%p;
  for(int i=1;i<=n;i++){
    for(int j=i+1;j<=n;j++)
      while(a[j][i]){ // 辗转相除, 把 a[j][i] 消成 0
        ll t=a[i][i]/a[j][i];
        for(int k=i;k<=n;k++)
          a[i][k]=(a[i][k]+(p-t)*a[j][k])%p;
        swap(a[i],a[j]);
        res=p-res;
      }
    if(a[i][i]==0)return 0;
    res=res*a[i][i]%p;
  }
  return res%p;
}
```

SolveLinear.h

Description: 高斯-约旦消元解实系数线性方程组 (n 个方程, m 个未知数). 下标从 1 开始: $a[i][1..m]$ 为第 i 个方程的系数, $a[i][m+1]$ 为常数项. 无解返回 -1; 否则返回秩 r, $x[1..m]$ 为一组解 ($r<m$ 时有无穷多解, 自由元取 0, 且 pos[1..r] 为各主元所在列). a 会被破坏.
Usage: `// 填好 a[1..n][1..m+1], 多组数据重新填即可`
`int r=gauss(n,m);`
`if(r==-1) puts("No Solution");`
`else if(r<m) puts("Infinite"); // 仍可用 x[1..m]`
`else ... // 唯一解 x[1..m]`
Time: $\mathcal{O}\left(nm\cdot\min(n,m)\right)$

3ae8f4, 26 lines

```
const int N=505; // 方程数, 未知数上限
const double eps=1e-9;
double a[N][N],x[N];
int pos[N];
int gauss(int n,int m){
  int r=1; // 当前行
  for(int c=1;c<=m&&r<=n;c++){
    int p=r; // 选第 c 列绝对值最大的主元
    for(int i=r+1;i<=n;i++)
      if(fabs(a[i][c])>fabs(a[p][c]))p=i;
    if(fabs(a[p][c])<eps)continue; // 自由元
    swap(a[p],a[r]);
    for(int i=1;i<=n;i++)
      if(i!=r&&a[i][c]!=0){
        double t=a[i][c]/a[r][c];
        for(int j=c;j<=m+1;j++)a[i][j]-=t*a[r][j];
      }
    pos[r++]=c;
  }
  r--; // 秩
  for(int i=r+1;i<=n;i++)
    if(fabs(a[i][m+1])>eps)return -1; // 0 = 非 0
  for(int j=1;j<=m;j++)x[j]=0;
  for(int i=1;i<=r;i++)x[pos[i]]=a[i][m+1]/a[i][pos[i]];
  return r;
}
```

SolveLinearBinary.h

Description: 异或方程组 (模 2 线性方程组), 用 bitset 做高斯消元. 下标从 1 开始: a[i] 的第 $1..m$ 位为系数, 第 $m+1$ 位为常数项. 无解返回 -1; 否则返回秩 r, $x[1..m]$ 为一组解 ($r<m$ 时有 2^{m-r} 组解, 自由元取 0). a 会被破坏. 需要 $m+1<N$.
Usage: `for(int i=1;i<=n;i++) a[i].reset(); // 多组数据清空`
`a[i][j]=1; a[i][m+1]=b_i; // 第 i 个方程`
`int r=gaussXor(n,m); // 解在 x[1..m]`
Time: $\mathcal{O}\left(nm\cdot\min(n,m)/64\right)$

5823c2, 21 lines

```
const int N=1005; // 方程数, 未知数上限
bitset<N>a[N];
int x[N],pos[N];
int gaussXor(int n,int m){
  int r=1; // 当前行
  for(int c=1;c<=m&&r<=n;c++){
    int p=r;
    while(p<=n&&!a[p][c])p++;
    if(p>n)continue; // 自由元
    swap(a[p],a[r]);
    for(int i=1;i<=n;i++)
      if(i!=r&&a[i][c])a[i]^=a[r];
    pos[r++]=c;
  }
  r--; // 秩
  for(int i=r+1;i<=n;i++)
    if(a[i][m+1])return -1; // 0 = 1
  for(int j=1;j<=m;j++)x[j]=0;
  for(int i=1;i<=r;i++)x[pos[i]]=a[i][m+1];
  return r;
}
```

MatrixInverse.h

Description: 高斯-约旦消元求实矩阵的逆. 下标从 1 开始, 把 $a[1..n][1..n]$ 右边拼上单位阵 $a[1..n][n+1..2n]$ 一起消元. 可逆返回 true 且 $a[1..n][1..n]$ 变为逆矩阵; 奇异返回 false. 模素数版本: 主元选任意非 0 元, 除法改为乘逆元 qpow(v, MOD-2), 所有运算取模. 模 p^k: 设 B 为模 p 的逆, 反复 $B=B(2I-AB)$, 每次模数指数 k 翻倍.
Usage: `// 填好 a[1..n][1..n], 多组数据重新填即可`
`if(inverse(n)) ... // 逆矩阵在 a[1..n][1..n]`
Time: $\mathcal{O}\left(n^3\right)$

4194bc, 23 lines

```
const int N=505; // 矩阵大小上限
double a[N][N*2];
bool inverse(int n){
  for(int i=1;i<=n;i++)
    for(int j=n+1;j<=2*n;j++)a[i][j]=(j-n==i);
  for(int i=1;i<=n;i++){
    int p=i; // 选绝对值最大的主元
    for(int j=i+1;j<=n;j++)
      if(fabs(a[j][i])>fabs(a[p][i]))p=j;
    if(fabs(a[p][i])<1e-12)return false;
    swap(a[i],a[p]);
    double t=1/a[i][i];
    for(int j=1;j<=2*n;j++)a[i][j]*=t;
    for(int k=1;k<=n;k++)
      if(k!=i&&a[k][i]!=0){
        double v=a[k][i];
        for(int j=1;j<=2*n;j++)a[k][j]-=v*a[i][j];
      }
  }
  for(int i=1;i<=n;i++)
    for(int j=1;j<=n;j++)a[i][j]=a[i][j+n];
  return true;
}
```

4.4 FFT, NTT, FWT

FastFourierTransform.h

Description: FFT 多项式乘法 (迭代版, 自写复数). 系数下标从 0 开始, a_i 为 x^i 的系数. fft(f, 1) 为正变换, fft(f, -1) 为逆变换 (已除以 n), f 的长度须为 2 的幂. conv(a, b) 返回 $c_k=\sum_{i+j=k}a_ib_j$ (四舍五入成整数), 长度 $|a|+|b|-1$. 精度: $(\sum a_i^2+\sum b_i^2)\log_2 n<9\cdot10^{14}$ 时安全 (随机数据可更大); 否则用 NTT 或 FFTMod.
Usage: `vector<ll> a={1,2}, b={1,3}; // 1+2x, 1+3x`
`vector<ll> c=conv(a,b); // c={1,5,6}`
Time: $\mathcal{O}(n\log n)$, $n=|a|+|b|$ ($n=2^{22}$ 约 1s)

906974, 39 lines

```
struct cp{double x,y;}; // 复数 x+yi
cp operator+(cp a,cp b){return {a.x+b.x,a.y+b.y};}
cp operator-(cp a,cp b){return {a.x-b.x,a.y-b.y};}
cp operator*(cp a,cp b){
  return {a.x*b.x-a.y*b.y,a.x*b.y+a.y*b.x};
}
const double PI=acos(-1.0);
void fft(vector<cp>&f,int t){ // t=1 正变换, t=-1 逆变换
  int n=f.size();
  for(int i=1,j=0;i<n;i++){ // 位逆序置换
    for(int k=n>>1;(j^=k)<k;k>>=1);
    if(i<j)swap(f[i],f[j]);
  }
  for(int len=1;len<n;len<<=1){
    vector<cp>w(len); // 单位根直接用 cos, sin 算, 精度高
    for(int j=0;j<len;j++)
      w[j]={cos(PI*j/len),t*sin(PI*j/len)};
    for(int i=0;i<n;i+=len<<1)
      for(int j=0;j<len;j++){
        cp u=f[i+j],v=w[j]*f[i+j+len];
        f[i+j]=u+v;f[i+j+len]=u-v;
      }
  }
  if(t==-1)for(auto&z:f)z.x/=n,z.y/=n;
}
vector<ll>conv(const vector<ll>&a,const vector<ll>&b){
  if(a.empty()||b.empty())return {};
  int m=a.size()+b.size()-1,n=1;
  while(n<m)n<<=1;
```

```
  vector<cp>f(n);
  for(int i=0;i<(int)a.size();i++)f[i].x=a[i];
  for(int i=0;i<(int)b.size();i++)f[i].y=b[i];
  fft(f,1);
  for(auto&z:f)z=z*z; // (a+bi)^2 的虚部为 2ab
  fft(f,-1);
  vector<ll>c(m);
  for(int i=0;i<m;i++)c[i]=llround(f[i].y/2);
  return c;
}
```

FastFourierTransformMod.h

Description: 任意模数多项式乘法 (拆系数 FFT, 4 次 FFT). 系数下标从 0 开始. 把每个系数拆成 $a = a_1 2^{15} + a_0$, 分别卷积后合并, 结果模 MOD. MOD 改成题目的模数即可 (须 $< 2^{30}$). 输入须在 $[0, MOD)$ 内. 精度: $n \log_2 n \cdot MOD < 8.6 \cdot 10^{14}$ 时安全 (实际可到 10^{16}).

Usage: `vector<ll> c=convMod(a,b); // c[k]=sum a[i]b[k-i] mod MOD`

Time: $\mathcal{O}(n \log n)$, $n = |a| + |b|$, 约为普通 FFT 的两倍

"FastFourierTransform.h" cc7a4a, 29 lines

```
const ll MOD=1000000007; // 模数, 按题目修改
vector<ll>convMod(const vector<ll>&a,const vector<ll>&b){
  if(a.empty()||b.empty())return {};
  int m=a.size()+b.size()-1,n=1;
  while(n<m)n<<=1;
  vector<cp>P(n),Q(n),X(n),Y(n);
  // P = a1 + i a0, Q = b1 + i b0
  for(int i=0;i<(int)a.size();i++)
    P[i]={double(a[i]>>15),double(a[i]&32767)};
  for(int i=0;i<(int)b.size();i++)
    Q[i]={double(b[i]>>15),double(b[i]&32767)};
  fft(P,1);fft(Q,1);
  for(int i=0;i<n;i++){
    int j=(n-i)&(n-1);
    cp s=P[i]+cp{P[j].x,-P[j].y}; // 2 FFT(a1)
    cp d=P[i]-cp{P[j].x,-P[j].y}; // 2i FFT(a0)
    cp A1={s.x/2,s.y/2},A0={d.y/2,-d.x/2};
    X[i]=A1*Q[i];Y[i]=A0*Q[i];
  }
  fft(X,-1);fft(Y,-1);
  // X = a1b1 + i a1b0, Y = a0b1 + i a0b0
  vector<ll>c(m);
  for(int i=0;i<m;i++){
    ll v11=llround(X[i].x)%MOD,v10=llround(X[i].y)%MOD;
    ll v01=llround(Y[i].x)%MOD,v00=llround(Y[i].y)%MOD;
    c[i]=((v11<<30)%MOD+((v10+v01)<<15)+v00)%MOD;
  }
  return c;
}
```

NumberTheoreticTransform.h

Description: NTT 多项式乘法, 模 $998244353 = 119 \cdot 2^{23} + 1$, 原根 $g = 3$. 系数下标从 0 开始, 输入须在 $[0, MOD)$ 内, 结果长度 $|a| + |b| - 1 \le 2^{23}$. ntt(f, 1) 正变换, ntt(f, -1) 逆变换 (已乘 $1/n$), f 长度为 2 的幂. 其他 NTT 模数 (原根都是 3): 469762049, 167772161, 1004535809. 任意模数用 FFTMod.

Usage: `vector<ll> a={1,2}, b={1,3};`
`vector<ll> c=nttConv(a,b); // c={1,5,6}`

Time: $\mathcal{O}(n \log n)$

fea475, 41 lines

```
const ll MOD=998244353,G=3;
ll qpow(ll a,ll b){
  ll r=1;
  for(a%=MOD;b;b>>=1,a=a*a%MOD)
    if(b&1)r=r*a%MOD;
  return r;
}
void ntt(vector<ll>&f,int t){ // t=1 正变换, t=-1 逆变换
  int n=f.size();
  for(int i=1,j=0;i<n;i++){ // 位逆序置换
    for(int k=n>>1;(j^=k)<k;k>>=1);
    if(i<j)swap(f[i],f[j]);
  }
  for(int len=1;len<n;len<<=1){
    ll wn=qpow(G,(MOD-1)/(len<<1));
    if(t==-1)wn=qpow(wn,MOD-2);
    for(int i=0;i<n;i+=len<<1){
      ll w=1;
      for(int j=0;j<len;j++,w=w*wn%MOD){
        ll u=f[i+j],v=w*f[i+j+len]%MOD;
        f[i+j]=(u+v)%MOD;
        f[i+j+len]=(u-v+MOD)%MOD;
      }
    }
  }
  if(t==-1){
    ll inv=qpow(n,MOD-2);
    for(auto&v:f)v=v*inv%MOD;
  }
}
vector<ll>nttConv(vector<ll>a,vector<ll>b){
  if(a.empty()||b.empty())return {};
  int m=a.size()+b.size()-1,n=1;
  while(n<m)n<<=1;
  a.resize(n);b.resize(n);
  ntt(a,1);ntt(b,1);
  for(int i=0;i<n;i++)a[i]=a[i]*b[i]%MOD;
  ntt(a,-1);
  a.resize(m);
  return a;
}
```

FWT.h

Description: 快速沃尔什变换 (FWT), 求位运算卷积 $c_k = \sum_{i \oplus j = k} a_i b_j$, $\oplus$ 为 or, and, xor. 下标从 0 开始, 长度 n 须为 2 的幂 (不足补 0), 两个数组等长. 值用 ll 精确计算 (不取模). $t = 1$ 正变换, $t = -1$ 逆变换. or 正变换即子集和 $\hat{a}_k = \sum_{i \subseteq k} a_i$; and 正变换为超集和. 取模: 每次加减后取模, xor 逆变换的 /2 改为乘 $inv2 = (MOD + 1)/2$.

Usage: `int n=1<<k; vector<ll> a(n),b(n); // 填好 a, b`
`vector<ll> c=xorConv(a,b); // c[k]=sum a[i]b[j], i xor j=k`
`vector<ll> d=orConv(a,b),e=andConv(a,b);`
`fwtOr(a,1); // 原地变为子集和, fwtOr(a,-1) 变回`

Time: $\mathcal{O}(n \log n)$

665944, 40 lines

```
void fwtOr(vector<ll>&a,int t){
  int n=a.size();
  for(int len=1;len<n;len<<=1)
    for(int i=0;i<n;i+=len<<1)
      for(int j=i;j<i+len;j++)a[j+len]+=t*a[j];
}
void fwtAnd(vector<ll>&a,int t){
  int n=a.size();
  for(int len=1;len<n;len<<=1)
    for(int i=0;i<n;i+=len<<1)
      for(int j=i;j<i+len;j++)a[j]+=t*a[j+len];
}
void fwtXor(vector<ll>&a,int t){
  int n=a.size();
  for(int len=1;len<n;len<<=1)
    for(int i=0;i<n;i+=len<<1)
      for(int j=i;j<i+len;j++){
        ll u=a[j],v=a[j+len];
        a[j]=u+v;a[j+len]=u-v;
        if(t==-1)a[j]/=2,a[j+len]/=2;
      }
}
vector<ll>orConv(vector<ll>a,vector<ll>b){
  fwtOr(a,1);fwtOr(b,1);
  for(int i=0;i<(int)a.size();i++)a[i]*=b[i];
  fwtOr(a,-1);
  return a;
}
vector<ll>andConv(vector<ll>a,vector<ll>b){
  fwtAnd(a,1);fwtAnd(b,1);
  for(int i=0;i<(int)a.size();i++)a[i]*=b[i];
  fwtAnd(a,-1);
  return a;
}
vector<ll>xorConv(vector<ll>a,vector<ll>b){
  fwtXor(a,1);fwtXor(b,1);
  for(int i=0;i<(int)a.size();i++)a[i]*=b[i];
  fwtXor(a,-1);
  return a;
}
```

MaxPlusConvolution.h

Description: (max, +) 卷积 $c_k = \max_{i+j=k}(a_i + b_j)$, 其中 b 是凹的 ($b_{j+1} - b_j$ 不增), a 任意. 下标从 0 开始, 结果长度 $|a| + |b| - 1$. b 凹时最优 i 关于 k 单调不降, 用分治 (决策单调性) 求解. 求 min 卷积时要求 b 是凸的: 全部取负即可.

Usage: `vector<ll> c=maxPlus(a,b); // b 须为凹, a, b 非空`

Time: $\mathcal{O}((|a| + |b|) \log(|a| + |b|))$

a61b91, 20 lines

```
vector<ll>va,vb,vc;
// 求 c[l..r], 已知最优的 i 在 [optl, optr] 中
void mpdc(int l,int r,int optl,int optr){
  if(l>r)return;
  int mid=(l+r)/2,best=-1;
  int lo=max(optl,mid-(int)vb.size()+1);
  int hi=min({optr,mid,(int)va.size()-1});
  for(int i=lo;i<=hi;i++){ // 取最左的最优 i
    ll v=va[i]+vb[mid-i];
    if(best==-1||v>vc[mid])vc[mid]=v,best=i;
  }
  mpdc(l,mid-1,optl,best);
  mpdc(mid+1,r,best,optr);
}
vector<ll>maxPlus(const vector<ll>&a,const vector<ll>&b){
  va=a;vb=b;
  vc.assign(a.size()+b.size()-1,0);
  mpdc(0,(int)vc.size()-1,0,(int)a.size()-1);
  return vc;
}
```

卷积用法

下面的代码直接调用上面的 conv (FFT), nttConv (NTT), xorConv 等 (FWT). 下标都从 0 开始.

- **多项式乘法 / 两数组卷积**: $c_k = \sum_i a_i b_{k-i}$, 长度 $|a|+|b|-1$. 差卷积 $\sum_i a_i b_{i+k}$ 把 b 反转.

```
// a[i], b[i] 为 x^i 的系数
vector<ll> c=nttConv(a,b); // 模 998244353, 先取模
vector<ll> d=conv(a,b); // 精确整数 (FFT), 可有负数
// 差卷积 e[k]=sum a[i]*b[i+k], k 从 -(n-1) 到 m-1:
vector<ll> rb(b.rbegin(),b.rend()); // m=b.size()
vector<ll> t=conv(a,rb); // e[k] 即 t[m-1-k]
```

- **大整数乘法**: 低位在前存数字, 卷积后进位.

```
string bigMul(string s,string t){ // 十进制, 无负号
  int n=s.size(),m=t.size();
  vector<ll> a(n),b(m); // 低位在前
  for(int i=0;i<n;i++)a[i]=s[n-1-i]-'0';
  for(int i=0;i<m;i++)b[i]=t[m-1-i]-'0';
```

```
  vector<ll> c=conv(a,b);
  string r;ll carry=0;
  for(int i=0;i<(int)c.size()||carry;i++){
    if(i<(int)c.size())carry+=c[i];
    r+=char('0'+carry%10),carry/=10;
  }
  while(r.size()>1&&r.back()=='0')r.pop_back();
  reverse(r.begin(),r.end());
  return r;
}
```

- **两数之和计数**: 求 $v_i + v_j = k$ $(i < j)$ 的对数, 值域 $[0, V]$. 自己卷自己得到有序对且含 $i = j$.

```
vector<ll> pairSum(vector<int>&v,int V){
  vector<ll> cnt(V+1);
  for(int x:v)cnt[x]++;
  vector<ll> f=conv(cnt,cnt); // 有序对, 含 i=j
  for(int x:v)f[2*x]--; // 去掉 i=j
  for(auto&y:f)y/=2; // 变成无序对 i<j
  return f; // f[k] 为对数, 长度 2V+1
}
```

- **带通配符的字符串匹配**: $\star$ 匹配任意字符. 字符值 1..26, 通配符为 0, 位置 i 匹配当且仅当 $\sum_j (p_j - s_{i+j})^2 p_j s_{i+j} = 0$. 展开后是三个卷积 ($p$ 反转). NTT 下和可能恰为模数的倍数, 误判概率极小; 介意可再换一个模数验证.

```
ll wv(char c){return c=='*'?0:c-'a'+1;} // 通配符为 0
vector<int> wildMatch(string s,string p){
  int n=s.size(),m=p.size();
  vector<int> res; // 匹配的起点, 下标从 0 开始
  if(m>n)return res;
  vector<ll> S[4],P[4]; // S[k][i] 为 s_i 的 k 次方
  for(int k=1;k<=3;k++)S[k].resize(n),P[k].resize(m);
  for(int i=0;i<n;i++){
    ll v=wv(s[i]);
    S[1][i]=v,S[2][i]=v*v,S[3][i]=v*v*v;
  }
  for(int j=0;j<m;j++){ // P 为 p 反转
    ll v=wv(p[m-1-j]);
    P[1][j]=v,P[2][j]=v*v,P[3][j]=v*v*v;
  }
  vector<ll> c1=nttConv(P[3],S[1]);
  vector<ll> c2=nttConv(P[2],S[2]);
  vector<ll> c3=nttConv(P[1],S[3]);
  for(int i=0;i+m<=n;i++){
    int t=i+m-1;
    if((c1[t]-2*c2[t]+c3[t]+2*MOD)%MOD==0)
      res.push_back(i);
  }
  return res;
}
```

- **位运算卷积**: 求 $v_i \oplus v_j = k$ $(i < j)$ 的对数 (值在 $[0, 2^K)$), 以及子集和.

```
vector<ll> pairXor(vector<int>&v,int K){
  vector<ll> cnt(1<<K);
  for(int x:v)cnt[x]++;
  vector<ll> f=xorConv(cnt,cnt); // 有序对, 含 i=j
  f[0]-=v.size(); // i=j 时 xor 为 0
  for(auto&y:f)y/=2;
  return f;
}
// or 同理: f=orConv(cnt,cnt), 对每个 x 做 f[x]--, 再除 2
// 子集和: g[S]=sum a[T], T 取遍 S 的子集
vector<ll> g=a;fwtOr(g,1); // fwtOr(g,-1) 还原
// 超集和: fwtAnd(g,1)
```

数论 (5)

5.1 模运算

ModularArithmetic.h

Description: 模质数 MOD 下的常用函数: 快速幂, 费马小定理求逆元, 预处理阶乘 fac 和阶乘逆元 ifac 求组合数. 加减: (a + b) % MOD, (a - b + MOD) % MOD.
Usage: `init(1000000); // 预处理 fac, ifac 到 n (n < N)`
`ll c = C(5, 2); // 10, m < 0 或 m > n 时为 0`
`ll x = inv(3); // 3 的逆元, 要求 MOD 是质数`
Time: init $\mathcal{O}(N)$, C $\mathcal{O}(1)$, qpow 和 inv $\mathcal{O}(\log MOD)$

3cacb3, 19 lines

```
const ll MOD=998244353; // 或 1e9+7, 必须是质数
const int N=1e6+5;
ll fac[N],ifac[N];
ll qpow(ll a,ll b){ // a 的 b 次方模 MOD
  ll r=1;a%=MOD;if(a<0)a+=MOD;
  for(;b;b>>=1,a=a*a%MOD)if(b&1)r=r*a%MOD;
  return r;
}
ll inv(ll a){return qpow(a,MOD-2);} // a 不能是 MOD 的倍数
void init(int n){
  fac[0]=1;
  for(int i=1;i<=n;i++)fac[i]=fac[i-1]*i%MOD;
  ifac[n]=inv(fac[n]);
  for(int i=n;i>=1;i--)ifac[i-1]=ifac[i]*i%MOD;
}
ll C(int n,int m){
  if(m<0||m>n)return 0;
  return fac[n]*ifac[m]%MOD*ifac[n-m]%MOD;
}
```

ModInverse.h

Description: 线性求 $1..n$ 模质数 p 的逆元, 要求 $n < p$. 原理: $p = \lfloor p/i \rfloor \cdot i + (p \bmod i)$. 单个数的逆元用快速幂 a^{p-2} 或 exgcd.
Usage: `initInv(n, p); // 之后 inv[i] 为 i 的逆元, 1 到 n`
Time: $\mathcal{O}(N)$

ccdefc, 6 lines

```
const int N=1e6+5;
ll inv[N];
void initInv(int n,ll p){
  inv[1]=1;
  for(int i=2;i<=n;i++)inv[i]=(p-p/i)*inv[p%i]%p;
}
```

ModPow.h

Description: 快速幂, 求 $a^b \bmod p$. 要求 p^2 不超过 long long, 即 p 不超过约 $3 \cdot 10^9$; 更大的模数用 ModMulLL 的 powmod.
Usage: `ll r = qpow(a, b, p); // a 可以为负, b 至少为 0`
Time: $\mathcal{O}(\log b)$

2ed24f, 5 lines

```
ll qpow(ll a,ll b,ll p){
  ll r=1%p;a%=p;if(a<0)a+=p;
  for(;b;b>>=1,a=a*a%p)if(b&1)r=r*a%p;
  return r;
}
```

ModLog.h

Description: 离散对数. bsgs 求最小的 $x \geq 0$ 使 $c \cdot a^x \equiv b \pmod m$, 要求 $\gcd(a, m) = 1$. exbsgs 求最小的 $x \geq 0$ 使 $a^x \equiv b \pmod m$, a, m 任意: 不断用 $d = \gcd(a, m)$ 同除 a^x, b, m (若 $d \nmid b$ 无解), 记下系数 $c = \prod(a/d)$, 直到 a, m 互质, 再用 bsgs. 无解返回 -1. m 不超过约 10^9 (乘法不溢出).
Usage: `ll x = bsgs(a, b, m); // gcd(a, m) = 1`
`ll y = exbsgs(a, b, m); // 任意 a, m, 无解为 -1`
`// a 的阶, 即最小的 k 使 a^k = 1, 为 bsgs(a, 1, m, a) + 1`
Time: $\mathcal{O}\left(\sqrt{m}\right)$

4e5530, 25 lines

```
ll bsgs(ll a,ll b,ll m,ll c=1){
  a%=m,b%=m,c%=m;
  if(c==b)return 0;
  ll t=sqrtl(m)+1,at=1;
  unordered_map<ll,ll>h; // b * a^j 对应的最大的 j
  for(ll j=0,cur=b;j<t;j++,cur=cur*a%m)h[cur]=j,at=at*a%m;
  // 找 c * a^(i t) = b * a^j, 则 x = i t - j
  for(ll i=1,cur=c;i<=t;i++){
    cur=cur*at%m;
    if(h.count(cur))return i*t-h[cur];
  }
  return -1;
}
ll exbsgs(ll a,ll b,ll m){
  a%=m,b%=m;
  if(b==1%m)return 0;
  ll k=0,c=1;
  for(ll d;(d=__gcd(a,m))!=1;){
    if(b%d)return -1;
    b/=d,m/=d,k++,c=c*(a/d)%m;
    if(c==b)return k; // x = k
  }
  ll x=bsgs(a,b,m,c);
  return x<0?-1:x+k;
}
```

ModMulLL.h

Description: 模数 m 在 long long 范围内 (到约 $9 \cdot 10^{18}$) 的 乘法和快速幂, 用 __int128 防止溢出.
Usage: `ll x = mulmod(a, b, m); // a * b mod m, 要求 a, b 在 [0, m)`
`ll y = powmod(a, b, m); // a 的 b 次方 mod m`
Time: mulmod $\mathcal{O}(1)$, powmod $\mathcal{O}(\log b)$

ec1b53, 6 lines

```
ll mulmod(ll a,ll b,ll m){return (__int128)a*b%m;}
ll powmod(ll a,ll b,ll m){
  ll r=1%m;a%=m;
  for(;b;b>>=1,a=mulmod(a,a,m))if(b&1)r=mulmod(r,a,m);
  return r;
}
```

ModSqrt.h

Description: Cipolla 求二次剩余: 求 x 使 $x^2 \equiv n \pmod p$, p 为质数. 找 a 使 $w = a^2 - n$ 是非二次剩余, 则 $x = (a + \sqrt{w})^{(p+1)/2}$, 在 $\mathbb{F}_p[\sqrt{w}]$ 里算. 另一个解是 $p - x$. p 可到 10^{18}.
Usage: `ll x = cipolla(n, p); // 无解返回 -1, 有解返回较小的根`
Time: 期望 $\mathcal{O}(\log p)$

83358f, 26 lines

```
typedef __int128 L;
ll pw(ll a,ll b,ll p){
  ll r=1%p;a%=p;
  for(;b;b>>=1,a=(L)a*a%p)if(b&1)r=(L)r*a%p;
  return r;
}
ll W; // w = a^2 - n
// (x + y sqrt w) 乘上 (a + b sqrt w)
void cmul(ll&x,ll&y,ll a,ll b,ll p){
  ll nx=((L)x*a+(L)y*b%p*W)%p;
  y=((L)x*b+(L)y*a)%p,x=nx;
}
ll cipolla(ll n,ll p){
  n%=p;if(n<0)n+=p;
  if(n==0||p==2)return n;
  if(pw(n,(p-1)/2,p)!=1)return -1; // 欧拉判别法
  ll a=0;
  do a++,W=((L)a*a%p-n+p)%p;
  while(pw(W,(p-1)/2,p)!=p-1);
  ll x=1,y=0,bx=a,by=1;
  for(ll e=(p+1)/2;e;e>>=1){
    if(e&1)cmul(x,y,bx,by,p);
```

```
    cmul(bx,by,bx,by,p);
  }
  return min(x,p-x);
}
```

5.2 素性测试

FastEratosthenes.h

Description: 线性筛 (欧拉筛), 一次求出 $[1,n]$ 内的质数, 欧拉函数 φ 和莫比乌斯函数 μ. 每个合数只被它的最小质因子筛掉一次. 积性函数都可以这样筛: 分 $i \bmod p = 0$ 和不等于 0 两种情况.
Usage: `sieve(n); // n < N, 只调用一次`
`pri[1..cnt]` 为质数, `vis[x] = 0` 且 `x` 至少为 2 时 `x` 是质数
`phi[x], mu[x]` 对 1 到 n 有效
Memory: N = 1e7 时约 130MB, 不需要的数组可以删掉
Time: $\mathcal{O}(N)$

82addb, 18 lines

```
const int N=1e7+5;
int pri[N],cnt,phi[N],mu[N];
bool vis[N];
void sieve(int n){
  phi[1]=mu[1]=1;
  for(int i=2;i<=n;i++){
    if(!vis[i])pri[++cnt]=i,phi[i]=i-1,mu[i]=-1;
    for(int j=1;j<=cnt&&(ll)i*pri[j]<=n;j++){
      int x=i*pri[j];
      vis[x]=1;
      if(i%pri[j]==0){ // pri[j] 是 i 的最小质因子
        phi[x]=phi[i]*pri[j],mu[x]=0;
        break;
      }
      phi[x]=phi[i]*(pri[j]-1),mu[x]=-mu[i];
    }
  }
}
```

MillerRabin.h

Description: 确定性 Miller-Rabin 素性测试, 用前 12 个质数作底, 对 long long 范围内所有数都正确. 乘法用 __int128.
Usage: `bool p = isPrime(n); // n 为 long long`
Time: 约 12 次快速幂, $\mathcal{O}(12\log n)$

a4b9c8, 21 lines

```
ll mulmod(ll a,ll b,ll m){return (__int128)a*b%m;}
ll powmod(ll a,ll b,ll m){
  ll r=1%m;a%=m;
  for(;b;b>>=1,a=mulmod(a,a,m))if(b&1)r=mulmod(r,a,m);
  return r;
}
bool isPrime(ll n){
  if(n<3||n%2==0)return n==2;
  ll d=n-1;int s=0;
  while(d%2==0)d/=2,s++; // n - 1 = d * 2^s
  for(ll a:{2,3,5,7,11,13,17,19,23,29,31,37}){
    if(n==a)return 1;
    ll x=powmod(a,d,n);
    if(x==1||x==n-1)continue;
    bool ok=0;
    for(int i=1;i<s&&!ok;i++)
      x=mulmod(x,x,n),ok=x==n-1;
    if(!ok)return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho 质因数分解. rho 用 $f(x) = x^2 + c$ 的倍增找环, 每 127 步做一次 gcd. 结果含重复的质因子, 顺序任意.
Usage: `vector<ll> f; factor(n, f); sort(f.begin(), f.end());`
`// 2299 得到 11 11 19, n = 1 时为空`
Time: 期望 $\mathcal{O}\left(n^{1/4}\right)$

"MillerRabin.h" 46d95c, 21 lines

```
mt19937_64 prng(20261107);
ll rho(ll n){ // n 为合数, 返回 n 的一个因子, 可能是 n
  if(n%2==0)return 2;
  ll c=prng()%(n-1)+1,s=0,t=0,v=1;
  for(int g=1;;g<<=1,s=t,v=1){
    for(int k=1;k<=g;k++){
      t=((__int128)t*t+c)%n;
      v=mulmod(v,abs(t-s),n);
      if(k%127==0){ll d=__gcd(v,n);if(d>1)return d;}
    }
    ll d=__gcd(v,n);
    if(d>1)return d;
  }
}
void factor(ll n,vector<ll>&f){
  if(n==1)return;
  if(isPrime(n)){f.push_back(n);return;}
  ll d=n;
  while(d==n)d=rho(n);
  factor(d,f),factor(n/d,f);
}
```

5.3 整除

SqrtIntervals.h

Description: 整除分块. $\lfloor n/i \rfloor$ 只有 $O(\sqrt{n})$ 种取值, 值为 $q = \lfloor n/l \rfloor$ 的一段是 $[l, \lfloor n/q \rfloor]$. 上取整: $q = \lceil n/l \rceil > 1$ 时段的右端是 $\lfloor (n-1)/(q-1) \rfloor$, $q = 1$ 时到 n. 两个数一起分块 (如 $\lfloor n/i \rfloor \lfloor m/i \rfloor$): $r = \min(n/(n/l), m/(m/l))$, $l \le \min(n, m)$. 例子求 $\sum_{i=1}^{n} \lfloor n/i \rfloor g(i)$, sg(l, r) 为 g 的区间和, 按题目改.
Usage: `ll s = floorSum(n); // sum floor(n/i) g(i), i 从 1 到 n`
`ll t = ceilSum(n); // sum ceil(n/i) g(i), i 从 1 到 n`
Time: $\mathcal{O}\left(\sqrt{N}\right)$

362a4f, 19 lines

```
ll sg(ll l,ll r){return r-l+1;} // g 的区间和, 这里 g = 1
ll floorSum(ll n){
  ll res=0;
  for(ll l=1,r;l<=n;l=r+1){
    ll q=n/l;
    r=n/q; // [l, r] 内 n/i 都等于 q
    res+=q*sg(l,r);
  }
  return res;
}
ll ceilSum(ll n){
  ll res=0;
  for(ll l=1,r;l<=n;l=r+1){
    ll q=(n+l-1)/l;
    r=q>1?(n-1)/(q-1):n; // [l, r] 内上取整都是 q
    res+=q*sg(l,r);
  }
  return res;
}
```

euclid.h

Description: 扩展欧几里得, 求 x, y 使 $ax + by = \gcd(a, b)$, 返回 gcd. 结果满足 $|x| \le |b|, |y| \le |a|$. 只要 gcd 用 __gcd(a, b). a, m 互质时 x 就是 a 模 m 的逆元 (调用 exgcd(a, m, x, y)). 解 $ax + by = c$: 有解当且仅当 $g|c$, 一组解为 $x \cdot c/g$; 通解 $x + k \cdot b/g$, 最小非负 x 对 b/g 取模.
Usage: `ll x, y, g = exgcd(a, b, x, y);`
`ll iv = (x % m + m) % m; // exgcd(a, m, x, y) 后, a 的逆元`
Time: $\mathcal{O}(\log \min(a, b))$

13745a, 6 lines

```
ll exgcd(ll a,ll b,ll&x,ll&y){
  if(!b){x=1,y=0;return a;}
  ll d=exgcd(b,a%b,y,x);
  y-=a/b*x;
  return d;
}
```

CRT.h

Description: 中国剩余定理, 解同余方程组 $x \equiv a_i \pmod{m_i}$, $i = 1..n$. crt 要求 m_i 两两互质: $x = \sum a_i M_i M_i^{-1}$, $M_i = M/m_i$. excrt 允许任意模数: 逐个合并, 设当前解为 $x \bmod M$, 解 $Mt \equiv a_i - x \pmod{m_i}$, 新解 $x + Mt$, 模数变为 $\mathrm{lcm}(M, m_i)$. 返回 $[0, \mathrm{lcm})$ 内的最小解, 无解返回 -1. 要求 lcm 不超过 long long.
Usage: `ll a[N], m[N]; // 下标从 1 开始, a[i] 可以为负`
`ll x = crt(a, m, n); // m 两两互质`
`ll y = excrt(a, m, n); // 任意 m, 无解为 -1`
Time: $\mathcal{O}(n \log M)$

"euclid.h" 74c863, 22 lines

```
ll crt(ll*a,ll*m,int n){
  ll M=1,x=0;
  for(int i=1;i<=n;i++)M*=m[i];
  for(int i=1;i<=n;i++){
    ll Mi=M/m[i],t,y,r=(a[i]%m[i]+m[i])%m[i];
    exgcd(Mi%m[i],m[i],t,y); // t 为 Mi 模 m[i] 的逆元
    t=(__int128)((t%m[i]+m[i])%m[i])*r%m[i];
    x=(x+(__int128)Mi*t)%M;
  }
  return x;
}
ll excrt(ll*a,ll*m,int n){
  ll x=0,M=1; // 当前 x mod M
  for(int i=1;i<=n;i++){
    ll t,y,g=exgcd(M,m[i],t,y),c=((a[i]-x)%m[i]+m[i])%m[i];
    if(c%g)return -1;
    ll mg=m[i]/g;
    t=(__int128)((t%mg+mg)%mg)*(c/g)%mg;
    x+=M*t,M*=mg; // 新的 x 小于新的 M
  }
  return x;
}
```

ExLucas.h

Description: 扩展 Lucas, 求 $\binom{n}{m} \bmod P$, P 不必是质数. 把 P 分解成 p^k, 对每个 p^k 去掉阶乘里所有的因子 p 再单独算 p 的次数, 最后用 CRT 合并. 与 p 互质的数模 p^k 的逆元用欧拉定理 $a^{\varphi(p^k)-1}$. 要求每个 p^k 不太大 (约 10^6 以内), n, m 可以到 10^{18}, P 不超过 10^{18}.
Usage: `ll c = exlucas(n, m, P); // m < 0 或 m > n 时返回 0`
Time: $\mathcal{O}\left(\sum p^k \log n\right)$

004094, 38 lines

```
ll pw(ll b,ll e,ll m){
  ll r=1%m;
  for(b%=m;e;e>>=1,b=b*b%m)
    if(e&1)r=r*b%m;
  return r;
}
ll facp(ll n,ll p,ll pk){ // n! 去掉所有因子 p 后模 pk
  if(!n)return 1;
  ll r=1;
  for(ll i=1;i<=pk;i++)if(i%p)r=r*i%pk;
  r=pw(r,n/pk,pk);
  for(ll i=1;i<=n%pk;i++)if(i%p)r=r*i%pk;
  return r*facp(n/p,p,pk)%pk;
}
ll Cpk(ll n,ll m,ll p,ll pk){
  ll e=0,ph=pk/p*(p-1); // e 为 C(n, m) 中因子 p 的次数
  for(ll t=n;t;t/=p)e+=t/p;
  for(ll t=m;t;t/=p)e-=t/p;
  for(ll t=n-m;t;t/=p)e-=t/p;
```

```
  ll r=facp(n,p,pk)*pw(facp(m,p,pk),ph-1,pk)%pk;
  r=r*pw(facp(n-m,p,pk),ph-1,pk)%pk;
  return r*pw(p,e,pk)%pk;
}
ll exlucas(ll n,ll m,ll P){
  if(m<0||m>n)return 0;
  ll x=0,M=1; // 已合并的解 x mod M
  for(ll p=2,Q=P;Q>1;p++){
    if(p*p>Q)p=Q;
    if(Q%p)continue;
    ll pk=1;
    while(Q%p==0)Q/=p,pk*=p;
    ll r=Cpk(n,m,p,pk),ph=pk/p*(p-1);
    // x + M t = r (mod pk), M 与 pk 互质
    ll t=(r-x%pk+pk)%pk*pw(M,ph-1,pk)%pk;
    x+=M*t,M*=pk;
  }
  return x;
}
```

5.3.1 Bézout 定理

对 $a \neq 0,\ b \neq 0,\ d = gcd(a,b)$ 是使下式有整数解的最小正整数

$$ax + by = d$$

若 (x,y) 是一组解, 则全部解为

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: 欧拉函数 $\varphi(n)$ 为 $1..n$ 中与 n 互质的数的个数. $n = p_1^{k_1} \cdots p_r^{k_r}$ 时 $\varphi(n) = n\prod_{p|n}(1-1/p)$; m,n 互质时 $\varphi(mn) = \varphi(m)\varphi(n)$. $\sum_{d|n}\varphi(d) = n$, $\sum_{1\le k\le n,\gcd(k,n)=1} k = n\varphi(n)/2$ $(n>1)$.
欧拉定理: a,n 互质 $\Rightarrow a^{\varphi(n)} \equiv 1 \pmod n$. **扩展欧拉定理**: $b \ge \varphi(n)$ 时 $a^b \equiv a^{b \bmod \varphi(n)+\varphi(n)} \pmod n$. **费马小定理**: p 为质数 $\Rightarrow a^p \equiv a \pmod p$.
下面的 getPhi 用试除法求单个 $\varphi(n)$; 要求 $1..n$ 所有的 φ 用 FastEratosthenes 的线性筛.
Usage: `ll f = getPhi(n); // n 至少为 1`
Time: $\mathcal{O}(\sqrt{n})$
24ae75, 9 lines

```
ll getPhi(ll n){
  ll r=n;
  for(ll i=2;i*i<=n;i++)if(n%i==0){
    r=r/i*(i-1);
    while(n%i==0)n/=i;
  }
  if(n>1)r=r/n*(n-1);
  return r;
}
```

5.4 分数

Fractions.h

Description: 分数, 分子分母为 long long, 总是约分且分母为正. 运算中 $p \cdot q$ 可能溢出: 保证分子分母不超过约 10^9, 或者把 ll 改成 __int128 (输出时转回 ll).
Usage: `Frac a = mk(1, 2), b = mk(1, 3); Frac c = a + b; // 5/6` `c.p, c.q 为分子分母; a < b, a == b 可直接比较`
Time: 每次运算 $\mathcal{O}$ (log)
05ad84, 18 lines

```
struct Frac{ll p,q;};
Frac mk(ll p,ll q){ // q 不能为 0
  if(q<0)p=-p,q=-q;
  ll g=gcd(p,q); // 非负, p = 0 时为 q
  return {p/g,q/g};
}
Frac operator+(Frac a,Frac b){
  return mk(a.p*b.q+b.p*a.q,a.q*b.q);
}
Frac operator-(Frac a,Frac b){
  return mk(a.p*b.q-b.p*a.q,a.q*b.q);
}
Frac operator*(Frac a,Frac b){return mk(a.p*b.p,a.q*b.q);}
Frac operator/(Frac a,Frac b){return mk(a.p*b.q,a.q*b.p);}
bool operator<(Frac a,Frac b){
  return (__int128)a.p*b.q<(__int128)b.p*a.q;
}
bool operator==(Frac a,Frac b){return a.p==b.p&&a.q==b.q;}
```

5.5 勾股数

勾股数可唯一地表示为

$$a = k\cdot(m^2-n^2),\ b = k\cdot(2mn),\ c = k\cdot(m^2+n^2),$$

其中 $m>n>0$, $k>0$, $m \perp n$, 且 m 或 n 为偶数.

5.6 素数

$p = 962592769$ 满足 $2^{21} \mid p-1$, 可能有用. 哈希可用 970592641 (31 位), 31443539979727 (45 位), 3006703054056749 (52 位). 小于 1 000 000 的素数有 78498 个.

除 $p=2, a>2$ 外, 模任意素数幂 p^a 都有原根, 共 $\phi(\phi(p^a))$ 个. $p=2, a>2$ 时, 群 $\mathbb{Z}_{2^a}^\times$ 同构于 $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 估计

$\sum_{d|n} d = O(n \log\log n)$.

n 的约数个数最多约为: $n < 5e4$ 时 100, $n < 1e7$ 时 500, $n < 1e10$ 时 2000, $n < 1e19$ 时 200 000.

5.8 Mobius 函数

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius 反演:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

其他常用形式:

$\sum_{d|n}\mu(d) = [n=1]$ (很有用)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d}\mu(d/n)g(d)$

$g(n) = \sum_{1\le m\le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1\le m\le n}\mu(m)g(\lfloor \frac{n}{m} \rfloor)$

组合数学 (6)

6.1 排列

6.1.1 阶乘

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

6.1.2 环

设 $g_S(n)$ 为所有环长都属于集合 S 的 n 元排列个数, 则

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

6.1.3 错排

没有任何元素在原位置上的排列.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

6.1.4 Burnside 引理

给定对称群 G 和集合 X, X 中在对称意义下不同的元素个数为

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

其中 X^g 是被 g 固定的元素 $(g.x = x)$.

若 $f(n)$ 统计长度为 n 的某种方案数, 取 $G = \mathbb{Z}_n$ 即可不计旋转对称, 得

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

6.2 划分与子集

6.2.1 划分函数

把 n 写成若干正整数之和的方案数, 不计顺序.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n-k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

6.2.2 Lucas 定理

设 n, m 为非负整数, p 为素数. 写成 $n = n_k p^k + ... + n_1 p + n_0$, $m = m_k p^k + ... + m_1 p + m_0$, 则 $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod p$.

6.2.3 二项式系数

multinomial.h

Description: 计算多项式系数 $\binom{k_1 + \cdots + k_n}{k_1, k_2, \ldots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \ldots k_n!}$. 逐个乘组合数, 每步都整除, 不取模; 结果和中间值 c * m 都要不超过 ll.
Usage: `ll c = multinomial({2, 1, 1}); // 4! / (2! 1! 1!) = 12`
Time: $\mathcal{O}(\sum k_i)$

2ffe05, 6 lines

```
ll multinomial(const vector<int>&k){
  ll c=1;int m=0; // m: 已放入的个数
  for(int x:k)
    for(int j=1;j<=x;j++)c=c*++m/j;
  return c;
}
```

6.3 常用计数数列

6.3.1 Bernoulli 数

Bernoulli 数的 EGF 为 $B(t) = \frac{t}{e^t - 1}$ (可用 FFT 求). $B[0, \ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

幂和:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1} \sum_{k=0}^{m} \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

无穷和的 Euler-Maclaurin 公式:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 第一类 Stirling 数

n 元排列中恰有 k 个环的排列数.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

6.3.3 Eulerian 数

恰有 k 个元素大于前一个元素的排列 $\pi \in S_n$ 个数. 等价于: 恰有 k 个 j 使 $\pi(j) > \pi(j+1)$; 恰有 $k+1$ 个 j 使 $\pi(j) \geq j$; 恰有 k 个 j 使 $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 第二类 Stirling 数

把 n 个不同元素恰好分成 k 组的方案数.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!} \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell 数

n 个不同元素的划分总数. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147$
对素数 p,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

6.3.6 有标号无根树

n 个点: n^{n-2}
连接 k 棵已有的树 (大小为 n_i): $n_1 n_2 \cdots n_k n^{k-2}$
度数为 d_i: $(n-2)!/((d_1-1)! \cdots (d_n-1)!)$

6.3.7 Catalan 数

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2} C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- $n \times n$ 网格中不越过对角线的单调路径.
- n 对括号组成的合法括号串.
- 有 $n+1$ 个叶子的二叉树 (每点 0 或 2 个儿子).
- 有 $n+1$ 个点的有序树.
- 连对角线把凸 $n+2$ 边形剖分成三角形的方案.
- 不含长为 3 的递增子序列的 $[n]$ 的排列.

图论 (7)

7.1 基础

FloydWarshall.h

Description: Floyd 全源最短路 (点 1..n), 允许负边权. 结束后 $d[i][j]$ 为最短距离, 不可达为 INF; 存在负环当且仅当 某个 $d[i][i] < 0$. 最后一段把经过负环的点对标成 −INF (不需要可删).
Usage: `init(n); d[u][v]=min(d[u][v],w); // 加边, 注意重边`
`floyd(n); if(d[u][v]==INF) 不可达; if(d[u][v]==-INF) 无下界`
Time: $\mathcal{O}(n^3)$

d5bfe9, 16 lines

```
const int N=505;
const ll INF=0x3f3f3f3f3f3f3f3fLL;
ll d[N][N];
void init(int n){
  for(int i=1;i<=n;i++)for(int j=1;j<=n;j++)
    d[i][j]=i==j?0:INF;
}
void floyd(int n){
  for(int k=1;k<=n;k++)for(int i=1;i<=n;i++)if(d[i][k]<INF)
    for(int j=1;j<=n;j++)if(d[k][j]<INF)
      d[i][j]=min(d[i][j],max(d[i][k]+d[k][j],-INF));
  // 负环: 能经过负环的点对距离为 -INF
  for(int k=1;k<=n;k++)if(d[k][k]<0)
    for(int i=1;i<=n;i++)if(d[i][k]<INF)
      for(int j=1;j<=n;j++)if(d[k][j]<INF)d[i][j]=-INF;
}
```

TopoSort.h

Description: Kahn 拓扑排序 (点 1..n). 返回是否为 DAG, 拓扑序在 ord[1..cnt]. 有环时 cnt $< n$, 环上及环能到达的点不会出现. 求字典序最小的拓扑序: 把 queue 换成小根堆 priority_queue.
Usage: `for(i=1..n) g[i].clear(); g[u].push_back(v); // 边 u 到 v`
`if(!topo(n)) 有环; 否则 ord[1..n] 为拓扑序`
Time: $\mathcal{O}(n+m)$

c1037a, 14 lines

```
const int N=2e5+5;
vector<int>g[N];
int deg[N],ord[N],cnt;
bool topo(int n){
  for(int i=1;i<=n;i++)deg[i]=0;
  for(int u=1;u<=n;u++)for(int v:g[u])deg[v]++;
  queue<int>q;cnt=0;
  for(int i=1;i<=n;i++)if(!deg[i])q.push(i);
  while(!q.empty()){
    int u=q.front();q.pop();ord[++cnt]=u;
    for(int v:g[u])if(--deg[v]==0)q.push(v);
  }
  return cnt==n;
}
```

DynamicConnectivity.h

Description: 离线动态图连通性: 线段树分治 + 可撤销并查集 (点 1..n, 时刻 1..q). 每条边在一个时刻区间 $[l, r]$ 内存在, 把它挂到线段树 上; solve 时进入结点加边, 离开时撤销. 到叶子 t 时并查集即为时刻 t 的图, 默认记录连通块个数 cc, 可在叶子处改成回答任意询问 (如 find(x)==find(y)). 第 i 个操作就是时刻 i: 边在时刻 a 加入, 时刻 b 删除, 则区间为 $[a, b-1]$; 到最后都没删的边区间为 $[a, q]$.
Usage: `init(n,q); // q 个时刻, 多组数据重新调用`
`map<pair<int,int>,int> st; // 边 (u<v) 的加入时刻`
`add(1,1,q,l,r,u,v); // 边 (u,v) 在时刻 [l,r] 存在`
`solve(1,1,q); // ans[t] 为时刻 t 的连通块数`
Time: $\mathcal{O}(m \log q \log n)$

8afee8, 39 lines

```
const int N=1e5+5,T=1e5+5; // N 点数, T 时刻数
int fa[N],siz[N],stk[N],tp,cc,ans[T];
vector<pair<int,int>>e[T<<2];
// 按大小合并, 不路径压缩才能撤销 ()
```

```
int find(int x){while(x!=fa[x])x=fa[x];return x;}
void merge(int x,int y){
  x=find(x),y=find(y);
  if(x==y)return;
  if(siz[x]<siz[y])swap(x,y);
  fa[y]=x,siz[x]+=siz[y],stk[++tp]=y,cc--;
}
void undo(int t){ // 撤销到栈大小为 t
  while(tp>t){
    int y=stk[tp--];
    siz[fa[y]]-=siz[y],fa[y]=y,cc++;
  }
}
void init(int n,int q){
  for(int i=1;i<=n;i++)fa[i]=i,siz[i]=1;
  for(int i=0;i<=q*4;i++)e[i].clear();
  cc=n,tp=0;
}
void add(int p,int l,int r,int ql,int qr,int u,int v){
  if(ql>qr)return;
  if(ql<=l&&r<=qr){e[p].push_back({u,v});return;}
  int mid=(l+r)/2;
  if(ql<=mid)add(p*2,l,mid,ql,qr,u,v);
  if(qr>mid)add(p*2+1,mid+1,r,ql,qr,u,v);
}
void solve(int p,int l,int r){
  int t=tp;
  for(auto[u,v]:e[p])merge(u,v);
  if(l==r)ans[l]=cc; // 在这里回答时刻 l 的询问
  else{
    int mid=(l+r)/2;
    solve(p*2,l,mid),solve(p*2+1,mid+1,r);
  }
  undo(t);
}
```

7.2 网络流

MinCostMaxFlow.h

Description: 最小费用最大流 (SPFA 找最短路, 逐条增广). 链式前向星, cnt 从 1 开始, 边 e 与反向边 $e\oplus 1$ 成对, 点编号 $1..n$. 允许负费用, 但不能有负环. 返回 (最大流, 最小费用). 输出方案: 原边 e 的流量为 cap[$e\oplus 1$]; 跑完后 dis[v] 小于 INF 的点 (残量网络中 s 能到达) 为最小割的 S 侧. 最小费用二分图匹配: 左 $\to$ 右且满流的边即为匹配边. 求最大费用: 费用取负, 答案再取负.

Usage: `init(n); // 多组数据每次调用`

```
int e=add(u,v,c,w); // 容量 c, 单位费用 w, 记下编号
auto [f,co]=mcmf(s,t); // 最大流, 最小费用
ll fe=cap[e^1]; // 边 e 上的流量
bool inS=dis[v]<INF; // v 在最小割的 S 侧
// 带权匹配: s 连左, 右连 t 容量 1 费用 0, 左右边容量 1
id[k]=add(i,L+j,1,w); // 若 cap[id[k]] 为 0 则第 k 条边被选中
```

Time: $\mathcal{O}(F\cdot VE)$, F 为最大流, 实际远快于此. e91921, 41 lines

```
const int N=5005,M=1e5+5; // 点数; 边数, 含反向边要开 2 倍
const ll INF=1e18;
int n,cnt=1,head[N],nxt[M],to[M],pre[N];
ll cap[M],cost[M],dis[N],fl[N]; // fl: 路径上的最小容量
bool inq[N];
void init(int _n){
  n=_n;cnt=1;
  for(int i=1;i<=n;i++)head[i]=0;
}
int add(int u,int v,ll c,ll w){ // 返回正向边编号
  to[++cnt]=v;cap[cnt]=c;cost[cnt]=w;
  nxt[cnt]=head[u];head[u]=cnt;
  to[++cnt]=u;cap[cnt]=0;cost[cnt]=-w;
  nxt[cnt]=head[v];head[v]=cnt;
  return cnt-1;
}
bool spfa(int s,int t){
  for(int i=1;i<=n;i++)dis[i]=INF,inq[i]=0;
  queue<int>q;q.push(s);dis[s]=0;fl[s]=INF;inq[s]=1;
  while(!q.empty()){
    int u=q.front();q.pop();inq[u]=0;
    for(int e=head[u];e;e=nxt[e]){
      int v=to[e];
      if(cap[e]&&dis[u]+cost[e]<dis[v]){
        dis[v]=dis[u]+cost[e];pre[v]=e;
        fl[v]=min(fl[u],cap[e]);
        if(!inq[v])inq[v]=1,q.push(v);
      }
    }
  }
  return dis[t]<INF;
}
pair<ll,ll>mcmf(int s,int t){
  ll flow=0,c=0;
  while(spfa(s,t)){
    ll f=fl[t];flow+=f;c+=f*dis[t];
    for(int v=t;v!=s;v=to[pre[v]^1])
      cap[pre[v]]-=f,cap[pre[v]^1]+=f;
  }
  return {flow,c};
}
```

Dinic.h

Description: 最大流 Dinic, 链式前向星存图, cnt 从 1 开始, 所以边 e 与反向边 $e\oplus 1$ 成对. 点编号 $1..n$. 输出方案: 原边 e 的流量为 cap[$e\oplus 1$] (反向边的剩余容量); 跑完后最后一次 bfs 失败, dep[v] 非 0 的点 (残量网络中 s 能到达) 就是最小割的 S 侧, 割边为 S 侧指向 T 侧的正向边. 二分图匹配: 左 $\to$ 右且满流的边即为匹配边. 无向边 (u,v,c) 用 add(u,v,c), add(v,u,c) 两条有向边.

Usage: `init(n); // 多组数据每次调用`

```
int e=add(u,v,c); // 返回正向边编号, 记下来输出方案
ll f=dinic(s,t);
ll fe=cap[e^1]; // 边 e 上的流量
bool inS=dep[v]; // v 在最小割的 S 侧
// 二分图匹配: 左 1..L, 右 L+1..L+R, s=L+R+1, t=L+R+2
init(L+R+2); add(s,i,1); id[k]=add(i,L+j,1); add(L+j,t,1);
dinic(s,t); // 若 cap[id[k]] 为 0 即满流, 则第 k 条边被选中
```

Time: $\mathcal{O}\left(V^2E\right)$, 单位容量 $\mathcal{O}\left(E\sqrt{E}\right)$, 二分图 $\mathcal{O}\left(E\sqrt{V}\right)$ 2f6494, 46 lines

```
const int N=1e4+5,M=2e5+5; // 点数; 边数, 含反向边要开 2 倍
const ll INF=1e18;
int n,S,T,cnt=1,head[N],nxt[M],to[M],dep[N],cur[N];
ll cap[M]; // 剩余容量
void init(int _n){
  n=_n;cnt=1;
  for(int i=1;i<=n;i++)head[i]=0;
}
int add(int u,int v,ll c){ // 有向边 u 到 v, 容量 c
  to[++cnt]=v;cap[cnt]=c;nxt[cnt]=head[u];head[u]=cnt;
  to[++cnt]=u;cap[cnt]=0;nxt[cnt]=head[v];head[v]=cnt;
  return cnt-1;
}
bool bfs(){ // 求出所有 s 可达点的层数, 不提前退出
  for(int i=1;i<=n;i++)dep[i]=0;
  queue<int>q;q.push(S);dep[S]=1;
  while(!q.empty()){
    int u=q.front();q.pop();
    for(int e=head[u];e;e=nxt[e])if(cap[e]&&!dep[to[e]]){
      dep[to[e]]=dep[u]+1;q.push(to[e]);
    }
  }
  return dep[T];
}
ll dfs(int u,ll f){
  if(u==T)return f;
  ll res=0;
  for(int&e=cur[u];e;e=nxt[e]){ // 当前弧优化
    int v=to[e];
    if(cap[e]&&dep[v]==dep[u]+1){
      ll d=dfs(v,min(f,cap[e]));
      cap[e]-=d;cap[e^1]+=d;res+=d;f-=d;
      if(!f)break;
    }
  }
  if(!res)dep[u]=0; // 此点已无法增广
  return res;
}
ll dinic(int s,int t){
  S=s;T=t;ll flow=0;
  while(bfs()){
    for(int i=1;i<=n;i++)cur[i]=head[i];
    flow+=dfs(S,INF);
  }
  return flow;
}
```

MinCut.h

Description: 输出 s-t 最小割方案 (配合 Dinic). 跑完 dinic(s,t) 后, 最后一次 bfs 失败, dep[v] 非 0 的点即残量网络中从 s 能到达的点, 构成最小割的 S 侧, 其余点为 T 侧. 割边为从 S 侧指向 T 侧的原边, 它们都满流, 原容量为 cap[$e\oplus 1$], 容量和等于最大流.

Usage: `ll f=dinic(s,t); // 先跑最大流`

```
vector<int> cut=minCut(); // 割边编号, 即 add 的返回值
for(int e:cut) cout<<to[e^1]<<' '<<to[e]<<endl; // 割边 u v
for(int v=1;v<=n;v++) if(dep[v]) ...; // v 在 S 侧
```

Time: $\mathcal{O}(E)$ "Dinic.h" 736b1f, 6 lines

```
vector<int>minCut(){ // 跑完 dinic 后调用
  vector<int>res;
  for(int e=2;e<=cnt;e+=2) // 只看正向边
    if(dep[to[e^1]]&&!dep[to[e]])res.push_back(e);
  return res;
}
```

匹配 (8)

HopcroftKarp.h

Description: 二分图最大匹配 Hopcroft-Karp. 左部点 $1..n$, 右部点 $1..m$, g[u] 存左部点 u 连向的右部点. 返回匹配数. mL[u] 为左部点 u 匹配的右部点, mR[v] 为右部点 v 匹配的左部点, 未匹配为 0. bfs 按层分组, dfs 只走下一层, 每轮找一批最短增广路.

Usage: `init(n,m); // 多组数据每次调用`

```
g[u].push_back(v); // 左 u 连右 v
int res=hk();
for(int u=1;u<=n;u++) if(mL[u]) cout<<u<<' '<<mL[u]<<endl;
```

Time: $\mathcal{O}\left(E\sqrt{V}\right)$ 37292f, 37 lines

```
const int N=1e5+5; // 左右部点数的最大值
int n,m,mL[N],mR[N],d[N];
vector<int>g[N];
void init(int _n,int _m){
  n=_n;m=_m;
  for(int i=1;i<=n;i++)g[i].clear();
}
bool dfs(int u){
  int t=d[u]+1;d[u]=0; // 本轮不再访问 u
  for(int v:g[u])
    if(!mR[v]||(d[mR[v]]==t&&dfs(mR[v]))){
      mL[u]=v;mR[v]=u;return true;
    }
```

```
  return false;
}
int hk(){
  for(int i=1;i<=n;i++)mL[i]=0;
  for(int i=1;i<=m;i++)mR[i]=0;
  int res=0;
  while(true){
    // d[u] 为左部点 u 的层数, 0 表示不可用
    queue<int>q;bool found=false;
    for(int i=1;i<=n;i++){
      d[i]=0;
      if(!mL[i])d[i]=1,q.push(i);
    }
    while(!q.empty()){
      int u=q.front();q.pop();
      for(int v:g[u]){
        if(!mR[v])found=true;
        else if(!d[mR[v]])d[mR[v]]=d[u]+1,q.push(mR[v]);
      }
    }
    if(!found)return res;
    for(int i=1;i<=n;i++)if(!mL[i])res+=dfs(i);
  }
}
```

MinimumVertexCover.h

Description: 二分图最小点覆盖 (König 定理), 大小等于最大匹配数. 先跑 hk(), 再从左部所有未匹配点出发走交错路 (左到右走任意边, 右到左走匹配边), 标记到达的点 visL, visR. 最小点覆盖 = 左部未标记的点 + 右部已标记的点; 其补集 (左部已标记 + 右部未标记) 为最大独立集.

Usage: `init(n,m); g[u].push_back(v); // 同 HopcroftKarp`
`vector<int> A,B; int k=cover(A,B); // 覆盖左部点 A, 右部点 B`

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

"HopcroftKarp.h" a7d498, 19 lines

```
bool visL[N],visR[N];
int cover(vector<int>&A,vector<int>&B){
  int res=hk();
  for(int i=1;i<=n;i++)visL[i]=0;
  for(int i=1;i<=m;i++)visR[i]=0;
  vector<int>st;
  for(int i=1;i<=n;i++)if(!mL[i])st.push_back(i),visL[i]=1;
  while(!st.empty()){
    int u=st.back();st.pop_back();
    for(int v:g[u])if(!visR[v]){
      visR[v]=1;int w=mR[v]; // v 一定已匹配
      if(!visL[w])visL[w]=1,st.push_back(w);
    }
  }
  A.clear();B.clear();
  for(int i=1;i<=n;i++)if(!visL[i])A.push_back(i);
  for(int i=1;i<=m;i++)if(visR[i])B.push_back(i);
  return res; // A.size()+B.size() 等于 res
}
```

WeightedMatching.h

Description: 带权二分图最优匹配 (KM / 匈牙利算法, 势能 + Dijkstra 式). 左部 1..n, 右部 1..m, 要求 $n \le m$ (否则转置). a[i][j] 为左 i 匹配右 j 的代价, 给每个左部点匹配不同的右部点, 使代价和最小. 返回最小代价, 左 i 匹配右 ans[i]. 没有的边设为很大的代价 (如 10^{12}). 最大权: 代价取负, 答案再取负. 最大权且允许不匹配: 代价设为 $-\max(w,0)$, 并补 0 让 $n=m$.

Usage: `n=..., m=...; a[i][j]=w; // 1-indexed, 多组数据直接覆盖 a`
`ll c=km(); // 左 i 匹配右 ans[i]`
`// 最大权完美匹配: a[i][j]=-w[i][j]; ll best=-km();`

Time: $\mathcal{O}\left(n^2m\right)$

ec5102, 30 lines

```
const int N=505; // n, m 的最大值
const ll INF=1e18; // 大于任意可能的代价和
int n,m,p[N],way[N],ans[N]; // p[j]: 右 j 匹配的左部点
ll a[N][N],u[N],v[N],minv[N];
bool used[N];
ll km(){
  for(int i=0;i<=n;i++)u[i]=0;
  for(int j=0;j<=m;j++)v[j]=0,p[j]=0;
  for(int i=1;i<=n;i++){
    p[0]=i;int j0=0; // 虚拟右部点 0
    for(int j=0;j<=m;j++)minv[j]=INF,used[j]=0;
    do{ // 找一条到未匹配右部点的最短增广路
      used[j0]=1;
      int i0=p[j0],j1=0;ll delta=INF;
      for(int j=1;j<=m;j++)if(!used[j]){
        ll cur=a[i0][j]-u[i0]-v[j];
        if(cur<minv[j])minv[j]=cur,way[j]=j0;
        if(minv[j]<delta)delta=minv[j],j1=j;
      }
      for(int j=0;j<=m;j++)
        if(used[j])u[p[j]]+=delta,v[j]-=delta;
        else minv[j]-=delta;
      j0=j1;
    }while(p[j0]);
    // 沿增广路翻转
    do{int j1=way[j0];p[j0]=p[j1];j0=j1;}while(j0);
  }
  for(int j=1;j<=m;j++)if(p[j])ans[p[j]]=j;
  return -v[0];
}
```

GeneralMatching.h

Description: 一般图最大匹配, 带花树 (Edmonds blossom). 点编号 1..n. 对每个未匹配点 bfs 找增广路, 遇到奇环就缩成一朵花 (并查集 fa). vis 为 1 表示偶点 (可入队), 为 2 表示奇点. 返回匹配数, mt[u] 为 u 匹配的点, 未匹配为 0.

Usage: `init(n); // 多组数据每次调用`
`add(u,v); // 无向边`
`int res=solve();`
`for(int u=1;u<=n;u++) if(mt[u]>u) cout<<u<<'`
`'<<mt[u]<<endl;`

Time: $\mathcal{O}\left(n^3\right)$

152416, 62 lines

```
const int N=1005,M=2e5+5; // 点数; 边数, 每条边存两次
int n,cnt,head[N],nxt[M],to[M];
int mt[N],pre[N],fa[N],vis[N],dfn[N],tim;
queue<int>q;
void init(int _n){
  n=_n;cnt=0;tim=0;
  for(int i=0;i<=n;i++)head[i]=mt[i]=dfn[i]=0;
}
void add(int u,int v){
  to[++cnt]=v;nxt[cnt]=head[u];head[u]=cnt;
  to[++cnt]=u;nxt[cnt]=head[v];head[v]=cnt;
}
int find(int x){return fa[x]==x?x:fa[x]=find(fa[x]);}
int lca(int x,int y){ // 两个偶点在交错树上的最近公共花
  tim++;x=find(x);y=find(y);
  while(dfn[x]!=tim){
    dfn[x]=tim;
    x=find(pre[mt[x]]);
    if(y)swap(x,y);
  }
  return x;
}
void blossom(int x,int y,int w){ // 把 x 到花根 w 的路径缩进花
  while(find(x)!=w){
    pre[x]=y;y=mt[x];
    if(vis[y]==2)vis[y]=1,q.push(y); // 花里的奇点变成偶点
    if(find(x)==x)fa[x]=w;
    if(find(y)==y)fa[y]=w;
    x=pre[y];
  }
}
bool aug(int s){ // 从未匹配点 s 找增广路
  for(int i=1;i<=n;i++)fa[i]=i,vis[i]=pre[i]=0;
  while(!q.empty())q.pop();
  q.push(s);vis[s]=1;
  while(!q.empty()){
    int u=q.front();q.pop();
    for(int e=head[u];e;e=nxt[e]){
      int v=to[e];
      if(find(u)==find(v)||vis[v]==2)continue;
      if(!vis[v]){
        vis[v]=2;pre[v]=u;
        if(!mt[v]){ // 找到增广路, 沿 pre 翻转
          for(int x=v,y;x;x=y){
            y=mt[pre[x]];mt[x]=pre[x];mt[pre[x]]=x;
          }
          return true;
        }
        vis[mt[v]]=1;q.push(mt[v]);
      }else{ // u, v 都是偶点, 出现奇环
        int w=lca(u,v);
        blossom(u,v,w);blossom(v,u,w);
      }
    }
  }
  return false;
}
int solve(){
  int res=0;
  for(int i=1;i<=n;i++)if(!mt[i]&&aug(i))res++;
  return res;
}
```

8.1 DFS 算法

AllRoots.h

Description: 换根 DP 模板 (树, 点 1..n, 带边权). 例子同时求两件事: ans[u] = $\sum_v dist(u,v)$, far[u] = $\max_v dist(u,v)$. dfs1 自底向上求子树信息 (以 1 为根); dfs2 自顶向下, 用父亲的 答案推出儿子的答案. 去掉儿子 v 的贡献: 可减的量 (和) 直接减, 不可减的量 (max) 记最大值和次大值, 或用前缀/后缀合并.

Usage: `for(i=1..n) g[i].clear(); // 多组数据`
`g[u].push_back({v,w}),g[v].push_back({u,w});`
`dfs1(1,0); ans[1]=f[1]; up[1]=0; dfs2(1,0);`
`far[u]=max(d1[u],up[u]);`

Time: $\mathcal{O}(n)$

23cb18, 24 lines

```
const int N=2e5+5;
int n,c1[N];
vector<pair<int,int>>g[N]; // {v,w}
// siz 子树大小, f 子树内各点到 u 的距离和, ans 全树距离和
// d1/d2 向下最长次长链/, c1 最长链所在儿子, up 向上最长链
ll siz[N],f[N],ans[N],d1[N],d2[N],up[N],far[N];
void dfs1(int u,int fa){
  siz[u]=1,f[u]=d1[u]=d2[u]=c1[u]=0;
  for(auto[v,w]:g[u])if(v!=fa){
    dfs1(v,u);
    siz[u]+=siz[v],f[u]+=f[v]+siz[v]*w;
    ll t=d1[v]+w;
    if(t>d1[u])d2[u]=d1[u],d1[u]=t,c1[u]=v;
    else if(t>d2[u])d2[u]=t;
  }
}
void dfs2(int u,int fa){
  for(auto[v,w]:g[u])if(v!=fa){
    // 根从 u 移到 v: v 子树内的点近 w, 其余点远 w
    ans[v]=ans[u]+(n-2*siz[v])*w;
    up[v]=w+max(up[u],c1[u]==v?d2[u]:d1[u]);
```

```
    dfs2(v,u);
  }
}
```

SCC.h

Description: Tarjan 求有向图强连通分量 (点 1..n). scc[u] 为 u 所在分量编号 (1..cnt), sz[i] 为分量 i 的点数. Tarjan 得到的编号是 逆拓扑序: 缩点后的边总是从编号大的分量指向编号小的分量, 所以按 $cnt, cnt-1, \ldots, 1$ 的顺序就是 DAG 的拓扑序. shrink 建缩点后的 DAG h[1..cnt] (可能有重边, 需要时去重).
Usage: `init(n); g[u].push_back(v);`
`for(int i=1;i<=n;i++) if(!dfn[i]) tarjan(i);`
`shrink(n); // 可选`
`for(int i=cnt;i>=1;i--) ... // 按拓扑序 DP`
Time: $\mathcal{O}(n+m)$

2c5879, 25 lines

```
const int N=2e5+5;
vector<int>g[N],h[N]; // h 为缩点后的 DAG
int dfn[N],low[N],stk[N],tp,tim;
int scc[N],sz[N],cnt;bool ins[N];
void tarjan(int u){
  dfn[u]=low[u]=++tim,stk[++tp]=u,ins[u]=1;
  for(int v:g[u]){
    if(!dfn[v])tarjan(v),low[u]=min(low[u],low[v]);
    else if(ins[v])low[u]=min(low[u],dfn[v]);
  }
  if(low[u]==dfn[u]){
    cnt++;int v;
    do v=stk[tp--],ins[v]=0,scc[v]=cnt,sz[cnt]++;
    while(v!=u);
  }
}
void shrink(int n){
  for(int u=1;u<=n;u++)for(int v:g[u])
    if(scc[u]!=scc[v])h[scc[u]].push_back(scc[v]);
}
void init(int n){
  for(int i=1;i<=n;i++)
    g[i].clear(),h[i].clear(),dfn[i]=sz[i]=0;
  tim=tp=cnt=0;
}
```

BiconnectedComponents.h

Description: 无向图 Tarjan (点 1..n, 边 1..m), 一次 DFS 同时求: (1) 割点 cut[u] 与点双连通分量 vb[1..vc] (每个点双的点集, 割点会 属于多个点双; 孤立点单独成一个点双). 根是割点当且仅当它在 DFS 树上有至少两个儿子. (2) 桥 bri[id] 与边双连通分量 eb[u] (编号 1..ec). 用边编号判断 是否走回父边, 所以重边能正确处理. 自环不影响结果. 只需要其中一个时, 删掉另一部分的栈和代码即可. 圆方树 T: 圆点 1..n, 第 i 个点双为方点 $n+i$, 向点双内每个点连边; 孤立点也有自己的方点 (只连它一个). 割点等价于圆方树上度数至少 2. 缩点树 E2: 每个边双一个点, 每条桥连一条边 (图不连通时为森林).
Usage: `init(n,m); // 第 i 条边 (u,v):`
`g[u].push_back({v,i}),g[v].push_back({u,i});`
`for(int i=1;i<=n;i++) if(!dfn[i]) tarjan(i,0,i);`
`buildRS(n); // 圆方树 T[1..n+vc]`
`buildBT(n); // 缩点树 E2[1..ec], 点 u 对应 eb[u]`
Time: $\mathcal{O}(n+m)$

c9933f, 43 lines

```
const int N=2e5+5,M=4e5+5;
vector<pair<int,int>>g[N]; // {v边编号,}
vector<int>T[N*2],E2[N]; // 圆方树, 边双缩点树
int dfn[N],low[N],tim,s1[N],t1,s2[N],t2;
bool cut[N],bri[M];
int vc,ec,eb[N];vector<int>vb[N];
// in 为进入 u 的边编号, rt 为本次 DFS 的根
void tarjan(int u,int in,int rt){
  dfn[u]=low[u]=++tim,s1[++t1]=s2[++t2]=u;
  int ch=0;
  for(auto[v,id]:g[u]){
    if(!dfn[v]){
      tarjan(v,id,rt),low[u]=min(low[u],low[v]);
      if(low[v]>dfn[u])bri[id]=1;
      if(low[v]>=dfn[u]){ // 弹出一个点双
        if(++ch>1||u!=rt)cut[u]=1;
        vb[++vc].clear();int x;
        do x=s1[t1--],vb[vc].push_back(x);while(x!=v);
        vb[vc].push_back(u);
      }
    }else if(id!=in)low[u]=min(low[u],dfn[v]);
  }
  if(u==rt){if(!ch)vb[++vc]={u};t1=0;}
  if(low[u]==dfn[u]){ // 弹出一个边双
    ec++;int x;
    do x=s2[t2--],eb[x]=ec;while(x!=u);
  }
}
void buildRS(int n){
  for(int i=1;i<=n+vc;i++)T[i].clear();
  for(int i=1;i<=vc;i++)for(int x:vb[i])
    T[n+i].push_back(x),T[x].push_back(n+i);
}
void buildBT(int n){ // 每条桥两个方向各遍历一次
  for(int i=1;i<=ec;i++)E2[i].clear();
  for(int u=1;u<=n;u++)for(auto[v,id]:g[u])
    if(bri[id])E2[eb[u]].push_back(eb[v]);
}
void init(int n,int m){
  for(int i=1;i<=n;i++)g[i].clear(),dfn[i]=cut[i]=0;
  for(int i=1;i<=m;i++)bri[i]=0;
  tim=t1=t2=vc=ec=0;
}
```

2sat.h

Description: 2-SAT (变量 1..n). 点 x 表示 x 为真, 点 $x+n$ 表示 x 为假. 子句 $a \vee b$ 连边 $\neg a \to b$, $\neg b \to a$, 再用 Tarjan 求 SCC. 无解当且仅当某个 x 与 $x+n$ 在同一个 SCC. 有解时取 scc 编号小的那一边 (Tarjan 编号是逆拓扑序, 编号小的在拓扑序后面).
Usage: `init(n); // 多组数据重新调用`
`add(x,1,y,0); // 子句: x 为真 或 y 为假`
`add(x,1,x,1); // 强制 x 为真`
`if(!solve()) 无解; else ans[1..n] 为一组解 (1 为真)`
Time: $\mathcal{O}(n+m)$

e69f47, 31 lines

```
const int N=2e5+5; // 变量数, 点数为 2N
int n,dfn[N*2],low[N*2],stk[N*2],tp,tim,scc[N*2],cnt,ans[N];
bool ins[N*2];vector<int>g[N*2];
void tarjan(int u){
  dfn[u]=low[u]=++tim,stk[++tp]=u,ins[u]=1;
  for(int v:g[u]){
    if(!dfn[v])tarjan(v),low[u]=min(low[u],low[v]);
    else if(ins[v])low[u]=min(low[u],dfn[v]);
  }
  if(low[u]==dfn[u]){
    cnt++;int v;
    do v=stk[tp--],ins[v]=0,scc[v]=cnt;while(v!=u);
  }
}
// 子句: (x 取值 a) 或 (y 取值 b), a,b 为 0/1
void add(int x,int a,int y,int b){
  g[a?x+n:x].push_back(b?y:y+n); // x 不取 a 则 y 取 b
  g[b?y+n:y].push_back(a?x:x+n); // y 不取 b 则 x 取 a
}
bool solve(){
  for(int i=1;i<=2*n;i++)if(!dfn[i])tarjan(i);
  for(int i=1;i<=n;i++){
    if(scc[i]==scc[i+n])return 0;
    ans[i]=scc[i]<scc[i+n];
  }
  return 1;
}
void init(int _n){
  n=_n,tim=tp=cnt=0;
  for(int i=1;i<=2*n;i++)g[i].clear(),dfn[i]=0;
}
```

EulerWalk.h

Description: Hierholzer 求欧拉路径/回路 (点 1..n, 边 1..m), 有向或 无向, 允许重边和自环. 存在条件: 无向图奇度点为 0 或 2 个 (2 个时从 奇度点出发); 有向图所有点出度等于入度, 或恰有一个点出度比入度大 1 (起点) 且一个点入度比出度大 1. 另外所有边必须连通: 跑完后若 len $\neq m+1$ 也无解. 只要回路时额外检查起点等于终点即可.
Usage: `init(n,1); // 1 有向, 0 无向`
`addE(u,v); // 依次加边, 边编号 1..m`
`if(euler()) path[1..m+1] 为点序列, ep[1..m] 为边编号`
Time: $\mathcal{O}(n+m)$

ad2b0a, 34 lines

```
const int N=1e5+5,M=2e5+5;
int n,m,dir,cur[N],deg[N],path[M],len,ep[M],le;
bool vis[M];vector<pair<int,int>>g[N]; // {v边编号,}
void init(int _n,int d){
  n=_n,dir=d,m=len=le=0;
  for(int i=1;i<=n;i++)g[i].clear(),cur[i]=deg[i]=0;
}
// 有向图 deg 为出度减入度, 无向图 deg 为度数
void addE(int u,int v){
  m++,vis[m]=0,g[u].push_back({v,m});
  if(dir)deg[u]++,deg[v]--;
  else g[v].push_back({u,m}),deg[u]++,deg[v]++;
}
void dfs(int u){
  while(cur[u]<(int)g[u].size()){
    auto[v,id]=g[u][cur[u]++];
    if(vis[id])continue;
    vis[id]=1,dfs(v),ep[++le]=id;
  }
  path[++len]=u;
}
bool euler(){
  int s=1,c=0;
  for(int i=n;i>=1;i--)if(g[i].size())s=i;
  for(int i=1;i<=n;i++){
    if(dir&&deg[i]==1)c++,s=i;
    else if(dir&&deg[i]!=0&&deg[i]!=-1)return 0;
    if(!dir&&deg[i]%2)c++,s=i;
  }
  if(c>(dir?1:2))return 0;
  dfs(s);
  reverse(path+1,path+len+1),reverse(ep+1,ep+le+1);
  return len==m+1;
}
```

FunctionalGraph.h

Description: 基环树森林 / 内向树: 每个点 i 恰有一条出边指向 $a[i]$ ($1 \le a[i] \le n$). cyc[1..cnt] 为各个环 (沿出边的顺序, cyc[k][j] 的下一个点为 cyc[k][(j+1)%len]). 对每个点 u: rt[u] 为 从 u 出发第一个到达的环上点, dep[u] 为到 rt[u] 的步数 (尾巴长度, 环上点为 0), bel[u] 为所在环编号, pos[u] 为 rt[u] 在环中的下标. 从 u 走 $k \ge$ dep[u] 步到达 cyc[bel[u]][(pos[u]+k-dep[u])%len].
Usage: `for(int i=1;i<=n;i++) cin>>a[i];`
`solve(n); // 多组数据直接重新调用`
`bool onCycle=(dep[u]==0); int len=cyc[bel[u]].size();`
Time: $\mathcal{O}(n)$

4e8301, 26 lines

```
const int N=2e5+5;
int a[N],vis[N],rt[N],dep[N],bel[N],pos[N],cnt,st[N],tp;
vector<int>cyc[N];
void solve(int n){
  cnt=0;
  for(int i=1;i<=n;i++)vis[i]=0;
```

```
  for(int i=1;i<=n;i++)if(!vis[i]){
    int u=i;tp=0;
    while(!vis[u])vis[u]=1,st[++tp]=u,u=a[u];
    if(vis[u]==1){ // 找到新环, u 在环上
      cyc[++cnt].clear();
      while(vis[u]==1){
        vis[u]=2,rt[u]=u,dep[u]=0,bel[u]=cnt;
        pos[u]=cyc[cnt].size(),cyc[cnt].push_back(u);
        u=a[u];
      }
    }
    // 倒着处理链上的点
    for(;tp;tp--){
      int v=st[tp];
      if(vis[v]==2)continue;
      vis[v]=2,rt[v]=rt[a[v]],dep[v]=dep[a[v]]+1;
      bel[v]=bel[a[v]],pos[v]=pos[a[v]];
    }
  }
}
```

8.2 树

CompressTree.h

Description: 虚树 (单调栈建法). 需要原树 (根为 1) 的 dfs 序 dfn[] 和 lca(x,y), 由别的模板提供. 给定 k 个互不相同的关键点 h[1..k], 建出包含关键点, 两两 LCA 和根 1 的虚树, 点数不超过 $2k$. vt[u] 为虚树中 u 的儿子 (父到子), 虚树边 (u,v) 代表原树一条链, 长度为 dep[v]-dep[u]. 用到的虚树点为 vs[1..vn], 根 1 不一定是关键点 (自己用数组标记关键点). 每次 build 会重新清空用到的 vt.

Usage:
```
// 每次询问:
for(int i=1;i<=k;i++) cin>>h[i],key[h[i]]=1;
build(h,k); dp(1); // 在 vt 上 DP
for(int i=1;i<=k;i++) key[h[i]]=0; // 清空标记
```

Time: $\mathcal{O}(k \log k)$ 次 lca

aad1ab, 24 lines

```
const int N=2e5+5;
extern int dfn[];int lca(int x,int y); // 用户提供
vector<int>vt[N];
int stk[N],tp,vs[N],vn;
bool cmp(int x,int y){return dfn[x]<dfn[y];}
void push(int x){vt[x].clear(),vs[++vn]=x,stk[++tp]=x;}
void build(int*h,int k){
  sort(h+1,h+k+1,cmp);
  tp=vn=0,push(1);
  for(int i=1;i<=k;i++)if(h[i]!=1){
    int l=lca(h[i],stk[tp]);
    if(l!=stk[tp]){
      // 栈顶不在 h[i] 到根的链上, 弹出并连边
      while(dfn[l]<dfn[stk[tp-1]])
        vt[stk[tp-1]].push_back(stk[tp]),tp--;
      if(l!=stk[tp-1]){
        int x=stk[tp--];
        push(l),vt[l].push_back(x);
      }else vt[l].push_back(stk[tp--]);
    }
    push(h[i]);
  }
  for(int i=1;i<tp;i++)vt[stk[i]].push_back(stk[i+1]);
}
```

HLD.h

Description: 重链剖分 (点 1..n, 根 rt). dfs1 求 fa, dep, siz, son (重儿子); dfs2 求 top (链顶), dfn (新编号), rnk (dfn 的逆). 每条重链 和每棵子树在 dfn 上都是连续区间, 路径拆成 $O(\log n)$ 段区间. 数据结构由用户提供: upd(l,r,v) 给 dfn 区间 $[l,r]$ 加 v, qry(l,r) 求区间和, 一般接到线段树上 (按 rnk 建树). 维护 max 等只需改 qryPath 的合并. 边权: 把边权放到深的那个点上, 路径操作最后 一段改为 [dfn[x]+1, dfn[y]] (x 等于 y 时跳过), 即不含 LCA.

Usage:
```
// 把 upd 和 qry 接到自己的线段树上, 例如:
void upd(int l,int r,ll v){update(1,1,n,l,r,v);}
ll qry(int l,int r){return query(1,1,n,l,r);}
// 多组数据: g[1..n] 清空, tim=0
dfs1(rt,0); dfs2(rt,rt); // 线段树第 i 个位置初值 a[rnk[i]]
updPath(x,y,v); qryPath(x,y); // 路径
updSub(x,v); qrySub(x); lca(x,y); // 子树, LCA
```

Time: 每次路径操作 $\mathcal{O}(\log^2 n)$, 子树操作 $\mathcal{O}(\log n)$

3df905, 42 lines

```
const int N=2e5+5;
void upd(int l,int r,ll v);ll qry(int l,int r); // 用户提供
vector<int>g[N];
int fa[N],dep[N],siz[N],son[N],top[N],dfn[N],rnk[N],tim;
void dfs1(int u,int f){
  fa[u]=f,dep[u]=dep[f]+1,siz[u]=1,son[u]=0;
  for(int v:g[u])if(v!=f){
    dfs1(v,u),siz[u]+=siz[v];
    if(siz[v]>siz[son[u]])son[u]=v;
  }
}
void dfs2(int u,int t){
  top[u]=t,dfn[u]=++tim,rnk[tim]=u;
  if(son[u])dfs2(son[u],t);
  for(int v:g[u])if(v!=fa[u]&&v!=son[u])dfs2(v,v);
}
int lca(int x,int y){
  while(top[x]!=top[y]){
    if(dep[top[x]]<dep[top[y]])swap(x,y);
    x=fa[top[x]];
  }
  return dep[x]<dep[y]?x:y;
}
void updPath(int x,int y,ll v){
  while(top[x]!=top[y]){
    if(dep[top[x]]<dep[top[y]])swap(x,y);
    upd(dfn[top[x]],dfn[x],v),x=fa[top[x]];
  }
  if(dep[x]>dep[y])swap(x,y);
  upd(dfn[x],dfn[y],v);
}
ll qryPath(int x,int y){
  ll res=0;
  while(top[x]!=top[y]){
    if(dep[top[x]]<dep[top[y]])swap(x,y);
    res+=qry(dfn[top[x]],dfn[x]),x=fa[top[x]];
  }
  if(dep[x]>dep[y])swap(x,y);
  return res+qry(dfn[x],dfn[y]);
}
void updSub(int x,ll v){upd(dfn[x],dfn[x]+siz[x]-1,v);}
ll qrySub(int x){return qry(dfn[x],dfn[x]+siz[x]-1);}
```

LinkCutTree.h

Description: LCT 动态树 (点 1..n), 支持连边, 删边, 判连通, 单点 改值, 路径求和. 每条实链用一棵 splay 维护 (中序为深度从小到大), fa[x] 对 splay 根表示虚边指向的父亲. split(x,y) 后 y 为 splay 根, sum[y] 即 x 到 y 路径和. 维护 max/xor 等改 pushup 即可.

Usage:
```
for(int i=1;i<=n;i++) val[i]=a[i]; init(n);
link(x,y); cut(x,y); // 不合法的操作会被忽略
connected(x,y); modify(x,v); // val[x] 改为 v
split(x,y); ll s=sum[y]; // 要求 x,y 连通
```

Time: 均摊 $\mathcal{O}(\log n)$

8882ae, 48 lines

```
const int N=3e5+5;
int ch[N][2],fa[N],rev[N],st[N];ll val[N],sum[N];
bool nrt(int x){return ch[fa[x]][0]==x||ch[fa[x]][1]==x;}
void pushup(int x){sum[x]=sum[ch[x][0]]+sum[ch[x][1]]+val[x];}
void pushr(int x){swap(ch[x][0],ch[x][1]),rev[x]^=1;}
void pushdown(int x){
  if(rev[x])pushr(ch[x][0]),pushr(ch[x][1]),rev[x]=0;
}
void rotate(int x){
  int y=fa[x],z=fa[y],k=ch[y][1]==x,w=ch[x][!k];
  if(nrt(y))ch[z][ch[z][1]==y]=x;
  ch[x][!k]=y,ch[y][k]=w;
  if(w)fa[w]=y;
  fa[y]=x,fa[x]=z,pushup(y);
}
void splay(int x){
  int y=x,tp=0;st[++tp]=y;
  while(nrt(y))st[++tp]=y=fa[y];
  while(tp)pushdown(st[tp--]);
  while(nrt(x)){
    y=fa[x];int z=fa[y];
    if(nrt(y))rotate((ch[y][0]==x)^(ch[z][0]==y)?x:y);
    rotate(x);
  }
  pushup(x);
}
void access(int x){
  for(int y=0;x;y=x,x=fa[x])splay(x),ch[x][1]=y,pushup(x);
}
void makert(int x){access(x),splay(x),pushr(x);}
int findrt(int x){
  access(x),splay(x);
  while(ch[x][0])pushdown(x),x=ch[x][0];
  splay(x);return x;
}
void split(int x,int y){makert(x),access(y),splay(y);}
bool connected(int x,int y){return findrt(x)==findrt(y);}
void link(int x,int y){makert(x);if(findrt(y)!=x)fa[x]=y;}
void cut(int x,int y){
  makert(x);
  if(findrt(y)==x&&fa[y]==x&&!ch[y][0])
    fa[y]=ch[x][1]=0,pushup(x);
}
void modify(int x,ll v){splay(x),val[x]=v,pushup(x);}
void init(int n){
  for(int i=1;i<=n;i++)
    ch[i][0]=ch[i][1]=fa[i]=rev[i]=0,sum[i]=val[i];
}
```

8.3 数学

8.3.1 生成树计数

建 $N \times N$ 矩阵 mat, 对每条边 $a \to b \in G$ 执行 mat[a][b]--, mat[b][b]++ (若 G 无向, 再执行 mat[b][a]--, mat[a][a]++). 删去第 i 行和对应列后求行列式, 即得以 i 为根的有向生成树个数 (若 G 无向, 删去任意一行及对应列即可).

8.3.2 Erdős–Gallai 定理

度数为 $d_1 \geq \cdots \geq d_n$ 的简单图存在, 当且仅当 $d_1 + \cdots + d_n$ 为偶数, 且对每个 $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

几何 (9)

9.1 几何基础

Point.h

Description: 平面点 / 向量, 几何部分的公共头文件 (其余文件都用它). 坐标类型由 T 决定, 默认 double; 整数坐标要求精确时改成 typedef ll T, 此时不要用 unit, rotate 和除法. sgn 按 eps 判符号, == 和 < 也带 eps. 多边形用 vector<Point> 存, 下标从 0 开始.

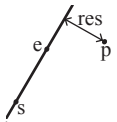
Usage:
```
Point a(1,2),b(3,4);
T c=cross(a,b),d=dot(a,b); // 叉积, 点积
double l=len(b-a); // 两点距离
if(sgn(cross(a,b,c))>0) // a,b,c 逆时针
vector<Point> poly; // 顶点 poly[0..n-1]
```
3a42e8, 26 lines

```
typedef double T; // 整数坐标可改为 ll
const double eps=1e-9,PI=acos(-1.0);
int sgn(double x){return x<-eps?-1:x>eps;}
struct Point{
  T x,y;
  Point(T x=0,T y=0):x(x),y(y){}
  Point operator+(Point b)const{return Point(x+b.x,y+b.y);}
  Point operator-(Point b)const{return Point(x-b.x,y-b.y);}
  Point operator*(T k)const{return Point(x*k,y*k);}
  Point operator/(T k)const{return Point(x/k,y/k);}
  bool operator==(Point b)const{
    return !sgn(x-b.x)&&!sgn(y-b.y);}
  bool operator<(Point b)const{ // 先比 x 再比 y
    return sgn(x-b.x)?x<b.x:sgn(y-b.y)<0;}
};
T dot(Point a,Point b){return a.x*b.x+a.y*b.y;}
T cross(Point a,Point b){return a.x*b.y-a.y*b.x;}
// (b-a) 叉乘 (c-a), 大于 0 表示 a,b,c 逆时针
T cross(Point a,Point b,Point c){return cross(b-a,c-a);}
T len2(Point a){return dot(a,a);} // 长度的平方
double len(Point a){return sqrt((double)len2(a));}
double angle(Point a){return atan2(a.y,a.x);} // 极角 [-pi,pi]
Point perp(Point a){return Point(-a.y,a.x);} // 逆时针转 90 度
Point unit(Point a){return a/len(a);} // 单位向量
Point rotate(Point a,double t){ // 绕原点逆时针转 t 弧度
  return Point(a.x*cos(t)-a.y*sin(t),a.x*sin(t)+a.y*cos(t));}
```

lineDistance.h

Description:

返回点 p 到直线 ab 的有向距离. 从 a 看向 b, 左侧为正, 右侧为负; a 与 b 重合时得 nan.



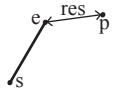
Usage:
```
double d=lineDist(a,b,p); // 正数表示 p 在左侧
double h=fabs(lineDist(a,b,p)); // 点到直线距离
```
"Point.h" 2d2735, 3 lines

```
double lineDist(Point a,Point b,Point p){
  return cross(a,b,p)/len(b-a);
}
```

SegmentDistance.h

Description:

返回点 p 到线段 se 的最短距离.



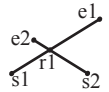
Usage:
```
Point s(0,0),e(2,2),p(1,1);
double d=segDist(s,e,p); // 得 0, 即 p 在线段上
```
"Point.h" c6b05b, 6 lines

```
double segDist(Point s,Point e,Point p){
  // 投影落在 s 外侧或 e 外侧时取端点距离
  if(s==e||sgn(dot(p-s,e-s))<0)return len(p-s);
  if(sgn(dot(p-e,s-e))<0)return len(p-e);
  return fabs((double)cross(s,e,p))/len(e-s);
}
```

SegmentIntersection.h

Description:

若线段 s1-e1 与 s2-e2 有唯一交点, 返回只含该点的 vector; 无交点返回空 vector; 有无穷多交点时返回 2 个元素, 即公共线段的两个端点 (按 x 再按 y 排序). T 为 ll 时若交点不是整点则结果错误. 中间有三个坐标的乘积, 注意溢出.



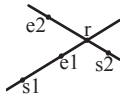
Usage:
```
vector<Point> v=segInter(s1,e1,s2,e2);
if(v.size()==1) // 唯一交点 v[0]
```
"Point.h", "OnSegment.h" 44c49c, 13 lines

```
vector<Point>segInter(Point a,Point b,Point c,Point d){
  T oa=cross(c,d,a),ob=cross(c,d,b);
  T oc=cross(a,b,c),od=cross(a,b,d);
  // 交点唯一且不是端点
  if(sgn(oa)*sgn(ob)<0&&sgn(oc)*sgn(od)<0)
    return {(a*ob-b*oa)/(ob-oa)};
  set<Point>s;
  if(onSegment(c,d,a))s.insert(a);
  if(onSegment(c,d,b))s.insert(b);
  if(onSegment(a,b,c))s.insert(c);
  if(onSegment(a,b,d))s.insert(d);
  return vector<Point>(s.begin(),s.end());
}
```

lineIntersection.h

Description:

若过 s1,e1 的直线与过 s2,e2 的直线有唯一交点, 返回 {1, 交点}; 无交点 (平行) 返回 {0, (0,0)}; 有无穷多交点 (重合) 返回 {-1, (0,0)}. T 为 ll 时若交点不是整点则结果错误. 中间有三个坐标的乘积, 注意溢出.



Usage:
```
pair<int,Point> r=lineInter(s1,e1,s2,e2);
if(r.first==1) // 交点为 r.second
```
"Point.h" 70c034, 7 lines

```
pair<int,Point>lineInter(Point s1,Point e1,Point s2,Point e2){
  T d=cross(e1-s1,e2-s2);
  if(sgn(d)==0) // 平行
    return {-(sgn(cross(s1,e1,s2))==0),Point(0,0)};
  T p=cross(s2,e1,e2),q=cross(s2,e2,s1);
  return {1,(s1*p+e1*q)/d};
}
```

sideOf.h

Description: 返回从 s 看向 e 时 p 所在的位置. 1/0/-1 $\Leftrightarrow$ 左侧/在直线上/右侧. 带参数 lim 的版本: p 到直线距离不超过 lim 时返回 0 (要求 $s \neq e$). 中间有乘积, 注意溢出.

Usage:
```
bool left=sideOf(p1,p2,q)==1; // q 在 p1 到 p2 左侧
int s=sideOf(p1,p2,q,1e-6); // 距离不超过 1e-6 算在线上
```
"Point.h" 01dfd6, 5 lines

```
int sideOf(Point s,Point e,Point p){return sgn(cross(s,e,p));}
int sideOf(Point s,Point e,Point p,double lim){
  double a=cross(s,e,p),l=len(e-s)*lim;
  return (a>l)-(a<-l);
}
```

OnSegment.h

Description: 当且仅当 p 在线段 se 上 (含端点) 时返回 true. 浮点坐标的误差已由 sgn 的 eps 处理.

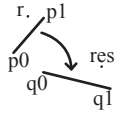
Usage:
```
bool on=onSegment(s,e,p);
```
"Point.h" d05f9c, 3 lines

```
bool onSegment(Point s,Point e,Point p){
  return sgn(cross(p,s,e))==0&&sgn(dot(s-p,e-p))<=0;
}
```

linearTransformation.h

Description:

对点 r 施加把直线 p0-p1 变到直线 q0-q1 的线性变换 (平移, 旋转和缩放), 要求 T 为 double.



Usage:
```
Point r2=linearTransformation(p0,p1,q0,q1,r);
```
"Point.h" 845944, 5 lines

```
Point linearTransformation(Point p0,Point p1,Point q0,
    Point q1,Point r){
  Point dp=p1-p0,dq=q1-q0,num(cross(dp,dq),dot(dp,dq));
  return q0+Point(cross(r-p0,num),dot(r-p0,num))/len2(dp);
}
```

Angle.h

Description: 精确的极角 (用 int 向量 (x, y) 加上绕原点的圈数 t 表示), 可排序, 适用于极角扫描. 比较顺序: 先比 t, 再比 $[0, 2\pi)$ 内的极角. 坐标绝对值不超过 10^9, 向量不能为 $(0, 0)$.

Usage:
```
vector<Angle> v; // v[0..n-1]
for(int i=0;i<n;i++)v.push_back(Angle(x[i],y[i]));
sort(v.begin(),v.end()); // 极角排序
// 双指针: 把每个角加一圈 (t360) 接在后面
for(int i=0;i<n;i++)v.push_back(v[i].t360());
for(int i=0,j=0;i<n;i++){
while(v[j]<v[i].t180())j++;
// v[i..j-1] 是极角在 [v[i], v[i]+pi) 内的角
}
```
1f7951, 32 lines

```
struct Angle{
  int x,y,t; // 向量 (x,y) 再加 t 圈
  Angle(int x,int y,int t=0):x(x),y(y),t(t){}
  Angle operator-(Angle b)const{return Angle(x-b.x,y-b.y,t);}
  int half()const{ // 角在 [pi, 2pi) 内时为 1
    assert(x||y);
    return y<0||(y==0&&x<0);
  }
  Angle t90()const{return Angle(-y,x,t+(half()&&x>=0));}
  Angle t180()const{return Angle(-x,-y,t+half());}
  Angle t360()const{return Angle(x,y,t+1);}
};
bool operator<(Angle a,Angle b){
  // 末尾再加上长度的平方, 可在极角相同时按长度排序
  return make_tuple(a.t,a.half(),a.y*(ll)b.x)<
    make_tuple(b.t,b.half(),a.x*(ll)b.y);
}
// 覆盖线段 ab 的最小角区间, 从 first 逆时针转到 second
pair<Angle,Angle>segmentAngles(Angle a,Angle b){
  if(b<a)swap(a,b);
  if(b<a.t180())return make_pair(a,b);
  return make_pair(b,a.t360());
}
Angle operator+(Angle a,Angle b){ // 点 a 加向量 b
  Angle r(a.x+b.x,a.y+b.y,a.t);
  if(a.t180()<r)r.t--;
  return r.t180()<a?r.t360():r;
}
Angle angleDiff(Angle a,Angle b){ // 角 b 减去角 a
  int tu=b.t-a.t;a.t=b.t;
  return Angle(a.x*b.x+a.y*b.y,a.x*b.y-a.y*b.x,tu-(b<a));
}
```

9.2 圆

CircleIntersection.h

Description: 求圆 $(a, r1)$ 与圆 $(b, r2)$ 的两个交点, 写入 out; 无交点时返回 false. 相切时两个交点相同. 两圆同心时要求 $r1 \neq r2$ (有 assert). 要求 T 为 double.

Usage: `pair<Point,Point> out;`
`if(circleInter(a,b,r1,r2,out)) // 交点 out.first, out.second`

"Point.h" 822dbd, 11 lines

```
bool circleInter(Point a,Point b,double r1,double r2,
    pair<Point,Point>&out){
  if(a==b){assert(r1!=r2);return false;}
  Point v=b-a;
  double d2=len2(v),sum=r1+r2,dif=r1-r2;
  double p=(d2+r1*r1-r2*r2)/(d2*2),h2=r1*r1-p*p*d2;
  if(sum*sum<d2||dif*dif>d2)return false;
  Point mid=a+v*p,per=perp(v)*sqrt(max(0.0,h2)/d2);
  out={mid+per,mid-per};
  return true;
}
```

CircleTangents.h

Description: 求两圆的外公切线; 把 r2 取负则求内公切线. 要求 T 为 double. 可能返回 0, 1 或 2 条切线 – 0 条: 一圆包含另一圆 (求内公切线时还包括两圆相交), 或两圆相同; 1 条: 两圆相切 (此时 .first = .second, 切线垂直于两圆心连线). .first 和 .second 分别是圆 1 和圆 2 上的切点. 求点到圆的切线时令 $r2 = 0$.

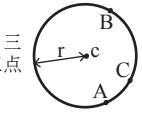
Usage: `vector<pair<Point,Point>> v=tangents(c1,r1,c2,r2);`
`// 圆 1 上切点 v[i].first, 圆 2 上切点 v[i].second`

"Point.h" 8de4a3, 14 lines

```
vector<pair<Point,Point>>tangents(Point c1,double r1,
    Point c2,double r2){
  Point d=c2-c1;
  double dr=r1-r2,d2=len2(d),h2=d2-dr*dr;
  if(sgn(d2)==0||sgn(h2)<0)return {};
  if(sgn(h2)==0)h2=0; // 相切, 只有一条
  vector<pair<Point,Point>>out;
  for(int s=-1;s<=1;s+=2){
    Point v=(d*dr+perp(d)*(sqrt(h2)*s))/d2;
    out.push_back({c1+v*r1,c2+v*r2});
  }
  if(h2==0)out.pop_back();
  return out;
}
```

circumcircle.h

Description:



三角形的外接圆是经过三个顶点的圆. ccRadius 返回过 A, B, C 三点的圆的半径, ccCenter 返回该圆的圆心. 要求 T 为 double, 三点不共线.

Usage: `double r=ccRadius(A,B,C);Point o=ccCenter(A,B,C);`

"Point.h" 1a3baa, 7 lines

```
double ccRadius(Point A,Point B,Point C){
  return len(B-A)*len(C-B)*len(A-C)/fabs(cross(B-A,C-A))/2;
}
Point ccCenter(Point A,Point B,Point C){
  Point b=C-A,c=B-A;
  return A+perp(b*len2(c)-c*len2(b))/cross(b,c)/2;
}
```

MinimumEnclosingCircle.h

Description: 最小圆覆盖 (随机增量法). 返回 {圆心, 半径}, 要求 T 为 double.

Usage: `vector<Point> ps; // ps[0..n-1], n 至少为 1`
`pair<Point,double> c=mec(ps); // 圆心 c.first, 半径 c.second`

Time: 期望 $\mathcal{O}(n)$

"circumcircle.h" 9fca95, 18 lines

```
pair<Point,double>mec(vector<Point>ps){
  shuffle(ps.begin(),ps.end(),mt19937(time(0)));
  int n=ps.size();
  Point o=ps[0];
  double r=0,E=1+1e-8; // 相对误差
  for(int i=0;i<n;i++)if(len(o-ps[i])>r*E){
    o=ps[i],r=0;
    for(int j=0;j<i;j++)if(len(o-ps[j])>r*E){
      o=(ps[i]+ps[j])/2;
      r=len(o-ps[i]);
      for(int k=0;k<j;k++)if(len(o-ps[k])>r*E){
        o=ccCenter(ps[i],ps[j],ps[k]);
        r=len(o-ps[i]);
      }
    }
  }
  return {o,r};
}
```

9.3 多边形

InsidePolygon.h

Description: 判断点 a 是否在多边形 p 内 (顺时针或逆时针, 可以是凹多边形). strict 为 true 时边界上的点返回 false. 中间有乘积, 注意溢出.

Usage: `vector<Point> p={Point(4,4),Point(1,2),Point(2,1)};`
`bool in=inPolygon(p,Point(3,3),false); // 含边界`

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h" 3a7f00, 10 lines

```
bool inPolygon(vector<Point>&p,Point a,bool strict=true){
  int cnt=0,n=p.size();
  for(int i=0;i<n;i++){
    Point q=p[(i+1)%n];
    if(onSegment(p[i],q,a))return !strict;
    // 射线法: 向右的水平射线与边相交的次数
    cnt^=((a.y<p[i].y)-(a.y<q.y))*cross(a,p[i],q)>0;
  }
  return cnt;
}
```

PolygonArea.h

Description: 返回多边形有向面积的两倍. 顶点按逆时针给出时为正, 顺时针为负. T 为 ll 时结果精确.

Usage: `vector<Point> v; // 顶点 v[0..n-1]`
`T a2=polygonArea2(v); // 面积为 fabs(a2)/2`

Time: $\mathcal{O}(n)$

"Point.h" cf9abe, 5 lines

```
T polygonArea2(vector<Point>&v){
  int n=v.size();T a=0;
  for(int i=0;i<n;i++)a+=cross(v[i],v[(i+1)%n]);
  return a;
}
```

PolygonCenter.h

Description: 返回多边形 (面积非 0) 的重心 (质心). 要求 T 为 double.

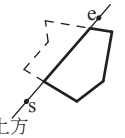
Usage: `vector<Point> v; // 顶点 v[0..n-1]`
`Point c=polygonCenter(v);`

Time: $\mathcal{O}(n)$

"Point.h" e0bc8f, 8 lines

```
Point polygonCenter(vector<Point>&v){
  Point res(0,0);double A=0;
  for(int i=0,j=v.size()-1;i<(int)v.size();j=i++){
    res=res+(v[i]+v[j])*cross(v[j],v[i]);
    A+=cross(v[j],v[i]);
  }
  return res/A/3;
}
```

PolygonCut.h



Description:
用从 s 到 e 的直线切割多边形, 切掉直线左侧的部分, 返回剩余多边形的顶点 (顺序不变). 要求 T 为 double.

Usage: `vector<Point> p; // 顶点 p[0..n-1]`
`p=polygonCut(p,Point(0,0),Point(1,0)); // 切掉 x 轴上方`

Time: $\mathcal{O}(n)$

"Point.h" bf28d4, 12 lines

```
vector<Point>polygonCut(vector<Point>&poly,Point s,Point e){
  vector<Point>res;
  int n=poly.size();
  for(int i=0;i<n;i++){
    Point cur=poly[i],prev=i?poly[i-1]:poly[n-1];
    double a=cross(s,e,cur),b=cross(s,e,prev);
    if((a<0)!=(b<0)) // 边与直线相交
      res.push_back(cur+(prev-cur)*(a/(a-b)));
    if(a<0)res.push_back(cur);
  }
  return res;
}
```

HalfPlane.h

Description: 半平面交 (极角排序 + 双端队列). 每条有向直线 $s \to e$ 表示它左侧的半平面. 返回交集凸多边形的顶点 (逆时针); 交集为空或面积为 0 时返回空 (面积很小时也可能返回退化的多边形). 要求交集有界: 不确定时先加入一个大矩形的 4 条边. 要求 T 为 double.

Usage: `vector<Line> L; // L[0..n-1]`
`L.push_back({Point(0,0),Point(1,0)}); // 保留 x 轴上方`
`vector<Point> poly=halfPlane(L);`

Time: $\mathcal{O}(n \log n)$

"Point.h" 8e801b, 30 lines

```
struct Line{Point s,e;}; // 有向直线 s 到 e
double ang(Line l){return angle(l.e-l.s);}
bool cmpLine(Line a,Line b){ // 按极角排序, 相同时靠左的在前
  if(sgn(ang(a)-ang(b)))return ang(a)<ang(b);
  return sgn(cross(a.s,a.e,b.s))<0;
}
bool onRight(Line l,Point p){return sgn(cross(l.s,l.e,p))<=0;}
Point inter(Line a,Line b){ // 两直线交点, 要求不平行
  Point u=a.e-a.s,v=b.e-b.s;
  return a.s+u*(cross(v,b.s-a.s)/cross(v,u));
}
vector<Point>halfPlane(vector<Line>L){
  sort(L.begin(),L.end(),cmpLine);
  int n=L.size(),h=0,t=-1;
  vector<Line>q(n);vector<Point>p(n); // 队列 q[h..t]
  for(int i=0;i<n;i++){
    // 同向的只留第一条, 即最靠左的
    if(i&&!sgn(ang(L[i])-ang(L[i-1])))continue;
    while(h<t&&onRight(L[i],p[t-1]))t--;
    while(h<t&&onRight(L[i],p[h]))h++;
    if(h<=t&&!sgn(cross(L[i].e-L[i].s,q[t].e-q[t].s)))
      return {}; // 与队尾反向平行, 交集为空
    q[++t]=L[i];
    if(h<t)p[t-1]=inter(q[t-1],q[t]);
  }
  while(h<t&&onRight(q[h],p[t-1]))t--;
  if(t-h<2)return {};
  p[t]=inter(q[t],q[h]);
  return vector<Point>(p.begin()+h,p.begin()+t+1);
}
```

ConvexHull.h



Description:
Andrew 单调链求凸包, 按逆时针顺序返回凸包顶点, 从最左下的点开始. 不保留共线点 (凸包边上的点不算顶点), 重复点只算一次. 所有点重合时返回 1 个点, 全部共线时返回 2 个端点.
Usage: `vector<Point> pts; // pts[0..n-1]`
`vector<Point> h=convexHull(pts); // h[0..m-1] 逆时针`
Time: $\mathcal{O}(n \log n)$

"Point.h" 570223, 18 lines

```
vector<Point>convexHull(vector<Point>p){
  sort(p.begin(),p.end());
  p.erase(unique(p.begin(),p.end()),p.end());
  int n=p.size(),k=0;
  if(n<=1)return p;
  vector<Point>h(2*n);
  // 要保留共线点, 两处改成严格小于 0, 全部共线时要特判
  for(int i=0;i<n;i++){ // 下凸壳
    while(k>=2&&sgn(cross(h[k-2],h[k-1],p[i]))<=0)k--;
    h[k++]=p[i];
  }
  for(int i=n-2,lo=k;i>=0;i--){ // 上凸壳
    while(k>lo&&sgn(cross(h[k-2],h[k-1],p[i]))<=0)k--;
    h[k++]=p[i];
  }
  h.resize(k-1); // 最后一个点是 p[0]
  return h;
}
```

HullDiameter.h

Description: 旋转卡壳求凸包直径 (最远点对), 返回距离的平方. 输入为逆时针凸包 (无重复点和共线点, 如 convexHull 的结果).
Usage: `vector<Point> h=convexHull(pts);`
`T d2=hullDiameter(h); // 直径为 sqrt(d2)`
Time: $\mathcal{O}(n)$

"Point.h" 320996, 12 lines

```
T hullDiameter(vector<Point>&h){
  int n=h.size();
  if(n<=2)return n==2?len2(h[0]-h[1]):0;
  T res=0;
  for(int i=0,j=1;i<n;i++){
    Point a=h[i],b=h[(i+1)%n];
    // h[j+1] 离边 ab 更远时移动 j
    while(sgn(cross(b-a,h[(j+1)%n]-h[j]))>0)j=(j+1)%n;
    res=max(res,max(len2(a-h[j]),len2(b-h[j])));
  }
  return res;
}
```

PointInsideHull.h

Description: 判断点 p 是否在凸包 l (逆时针, 无共线点) 内, 在内则返回 true. strict 为 true 时边界上的点不算在内. 凸包可以只有 1 或 2 个点.
Usage: `vector<Point> h=convexHull(pts);`
`bool in=inHull(h,p,false); // 含边界`
Time: $\mathcal{O}(\log n)$

"Point.h", "sideOf.h", "OnSegment.h" 401056, 13 lines

```
bool inHull(vector<Point>&l,Point p,bool strict=true){
  int a=1,b=l.size()-1,r=!strict;
  if(l.size()<3)return r&&onSegment(l[0],l.back(),p);
  if(sideOf(l[0],l[a],l[b])>0)swap(a,b);
  if(sideOf(l[0],l[a],p)>=r||sideOf(l[0],l[b],p)<=-r)
    return false;
  while(abs(a-b)>1){ // 二分 p 所在的扇形
    int c=(a+b)/2;
    if(sideOf(l[0],l[c],p)>0)b=c;
    else a=c;
  }
  return sgn(cross(l[a],l[b],p))<r;
}
```

LineHullIntersection.h

Description: 直线与凸多边形求交. 多边形须为逆时针且无共线点. lineHull(a, b, poly) 返回一对下标, 描述直线 ab 与多边形的相交情况: • $(-1,-1)$ 不相交, • $(i,-1)$ 只碰到顶点 i, • (i,i) 沿着边 $(i,i+1)$, • (i,j) 穿过边 $(i,i+1)$ 和 $(j,j+1)$. 最后一种情形中若穿过顶点 i, 视为发生在边 $(i,i+1)$ 上. 返回顺序与直线依次碰到多边形的顺序一致. extrVertex 返回凸多边形在方向 dir 上投影最大的顶点下标. 下标从 0 开始.
Usage: `array<int,2> r=lineHull(a,b,poly);`
`int i=extrVertex(poly,dir); // dir 方向上最远的顶点`
Time: $\mathcal{O}(\log n)$

"Point.h" afbf79, 41 lines

```
int extrVertex(vector<Point>&poly,Point dir){
  int n=poly.size(),lo=0,hi=n;
  // 比较顶点 i 和 j 在 dir 上的投影 (j 更远时为 1)
  auto cmp=[&](int i,int j){
    return sgn(cross(perp(dir),poly[i%n]-poly[j%n]));};
  auto extr=[&](int i){return cmp(i+1,i)>=0&&cmp(i,i-1+n)<0;};
  if(extr(0))return 0;
  while(lo+1<hi){
    int m=(lo+hi)/2;
    if(extr(m))return m;
    int ls=cmp(lo+1,lo),ms=cmp(m+1,m);
    if(ls<ms||(ls==ms&&ls==cmp(lo,m)))hi=m;
    else lo=m;
  }
  return lo;
}
array<int,2>lineHull(Point a,Point b,vector<Point>&poly){
  int n=poly.size();
  auto cmpL=[&](int i){return sgn(cross(a,poly[i],b));};
  int endA=extrVertex(poly,perp(a-b));
  int endB=extrVertex(poly,perp(b-a));
  if(cmpL(endA)<0||cmpL(endB)>0)return {-1,-1};
  array<int,2>res;
  for(int i=0;i<2;i++){
    int lo=endB,hi=endA;
    while((lo+1)%n!=hi){
      int m=((lo+hi+(lo<hi?0:n))/2)%n;
      if(cmpL(m)==cmpL(endB))lo=m;
      else hi=m;
    }
    res[i]=(lo+!cmpL(hi))%n;
    swap(endA,endB);
  }
  if(res[0]==res[1])return {res[0],-1};
  if(!cmpL(res[0])&&!cmpL(res[1]))
    switch((res[0]-res[1]+n+1)%n){
      case 0:return {res[0],res[0]};
      case 2:return {res[1],res[1]};
    }
  return res;
}
```

9.4 其他点集问题

ClosestPair.h

Description: 平面最近点对 (按 y 扫描 + set 维护 x 坐标在窗口内的点).
Usage: `vector<Point> v; // v[0..n-1], n 至少为 2`
`pair<Point,Point> pq=closest(v);`
`T d2=len2(pq.first-pq.second); // 最近距离的平方`
Time: $\mathcal{O}(n \log n)$

"Point.h" 591177, 17 lines

```
pair<Point,Point>closest(vector<Point>v){
  assert(v.size()>1);
  set<Point>S; // 按 x 排序
  sort(v.begin(),v.end(),[](Point a,Point b){
    return a.y<b.y;});
  T best=LLONG_MAX;pair<Point,Point>ret;
  int j=0;
  for(Point p:v){
    Point d(1+(ll)sqrt((double)best),0); // 当前答案的上界
    while(v[j].y<=p.y-d.x)S.erase(v[j++]);
    auto lo=S.lower_bound(p-d),hi=S.upper_bound(p+d);
    for(;lo!=hi;lo++)if(len2(*lo-p)<best)
      best=len2(*lo-p),ret={*lo,p};
    S.insert(p);
  }
  return ret;
}
```

字符串 (10)

KMP.h

Description: KMP. 串均为 1-indexed. nxt[i] 为 t[1..i] 的最长真 border 长度, 即既是前缀又是后缀, 且不等于 t[1..i] 本身的最长串. 例: abacaba 得 0 0 1 0 1 2 3. kmp 求出 t 在 s 中所有出现的起点.
Usage: `scanf("vector<int> occ=kmp(s,n,t,m); // 起点, 升序; 同时求出 nxt`
`getnxt(t,m); // 只要前缀函数时`
`// 最小周期: m-nxt[m]`
Time: $\mathcal{O}(n+m)$

586e27, 21 lines

```
const int N=1e6+5; // 串长上限
int nxt[N];
void getnxt(char*t,int m){
  nxt[1]=0;
  for(int i=2,j=0;i<=m;i++){
    while(j&&t[i]!=t[j+1])j=nxt[j];
    if(t[i]==t[j+1])j++;
    nxt[i]=j;
  }
}
// t 在 s 中出现的所有起点
vector<int>kmp(char*s,int n,char*t,int m){
  getnxt(t,m);
  vector<int>res;
  for(int i=1,j=0;i<=n;i++){
    while(j&&s[i]!=t[j+1])j=nxt[j];
    if(s[i]==t[j+1])j++;
    if(j==m)res.push_back(i-m+1),j=nxt[j];
  }
  return res;
}
```

Zfunc.h

Description: Z 函数, 即扩展 KMP. 串均为 1-indexed. z[i] 为 s[i..n] 与 s 的最长公共前缀长度, 约定 z[1] = n. (abacaba 得 7 0 1 0 3 0 1) exkmp: ext[i] 为 s[i..n] 与 t[1..m] 的最长公共前缀长度.
Usage: `getz(s,n); // z[1..n]`
`getz(t,m);exkmp(s,n,t,m); // ext[1..n], 会用到 t 的 z`
`// ext[i]==m 即 t 在 s 的位置 i 出现`
Time: $\mathcal{O}(n+m)$

f1fca5, 19 lines

```
const int N=1e6+5; // 串长上限
int z[N],ext[N];
void getz(char*s,int n){
  z[1]=n;
  for(int i=2,l=0,r=0;i<=n;i++){
    z[i]=i<=r?min(z[i-l+1],r-i+1):0;
    while(i+z[i]<=n&&s[1+z[i]]==s[i+z[i]])z[i]++;
    if(i+z[i]-1>r)l=i,r=i+z[i]-1;
  }
}
```

```
// 需先 getz(t,m)
void exkmp(char*s,int n,char*t,int m){
  for(int i=1,l=0,r=0;i<=n;i++){
    ext[i]=i<=r?min(z[i-l+1],r-i+1):0;
    while(i+ext[i]<=n&&ext[i]<m&&s[i+ext[i]]==t[1+ext[i]])
      ext[i]++;
    if(i+ext[i]-1>r)l=i,r=i+ext[i]-1;
  }
}
```

Manacher.h

Description: Manacher. 在 s[1..n] 的字符间插入 # 得到 a[1..m] ($m=2n+1$): a[2i] = s[i], 奇数位为 #. p[k] 为 a 中以 k 为中心的最长回文半径, 含中心; 对应原串中的回文长度为 p[k]-1, 起点为 (k-p[k])/2+1. 以 s[i] 为中心的最长奇回文长 p[2i]-1, 以 s[i], s[i+1] 之间为中心的最长偶回文长 p[2i+1]-1.
Usage: `int len=manacher(s,n); // 最长回文子串长度`
`// 多组数据直接再调用`
Time: $\mathcal{O}(n)$

b8890d, 15 lines

```
const int N=1e6+5; // 串长上限
char a[N*2];int p[N*2];
int manacher(char*s,int n){
  int m=0;a[0]='$';a[++m]='#';
  for(int i=1;i<=n;i++)a[++m]=s[i],a[++m]='#';
  a[m+1]='@'; // 两端哨兵, 不能在 s 中出现
  int ans=0;
  for(int i=1,mid=0,r=0;i<=m;i++){
    p[i]=i<r?min(p[2*mid-i],r-i):1;
    while(a[i+p[i]]==a[i-p[i]])p[i]++;
    if(i+p[i]>r)r=i+p[i],mid=i;
    ans=max(ans,p[i]-1);
  }
  return ans;
}
```

MinRotation.h

Description: 最小表示法: 求 s[1..n] 字典序最小的循环同构串的起点. 若有多个起点, 返回最小的.
Usage: `int k=minrot(s,n); // 最小表示为 s[k..n]+s[1..k-1]`
Time: $\mathcal{O}(n)$

0a36bd, 12 lines

```
int minrot(char*s,int n){
  // i, j 为两个候选起点, 从 0 开始; k 为已匹配长度
  int i=0,j=1,k=0;
  while(i<n&&j<n&&k<n){
    int x=s[(i+k)%n+1],y=s[(j+k)%n+1];
    if(x==y){k++;continue;}
    if(x>y)i+=k+1;else j+=k+1;
    if(i==j)j++;
    k=0;
  }
  return min(i,j)+1;
}
```

SuffixArray.h

Description: 后缀数组, 倍增 + 基数排序. 串 s[1..n] 为 1-indexed. sa[i] 为排名第 i 的后缀的起点, rk[i] 为后缀 i 的排名, 都从 1 开始. height[i] 为后缀 sa[i] 与 sa[i-1] 的最长公共前缀, height[1] = 0. 任意两后缀 i, j 的 LCP 为 height[rk[i]+1..rk[j]] 的最小值 ($rk_i < rk_j$, 配 RMQ). 字符需在 1..127 内, 否则改 m 为字符集大小并映射到 1..m.
Usage: `scanf("getsa(s,n);getheight(s,n);`
`// 本质不同子串数: n(n+1)/2 - sum height[i]`
`// 多组数据直接再调用`
Time: $\mathcal{O}(n \log n)$

f4960c, 40 lines

```
const int N=1e6+5; // 串长上限
int sa[N],rk[N*2],tp[N*2],cnt[N],height[N];
void getsa(char*s,int n){
  int m=127; // 字符集大小
  for(int i=1;i<=2*n;i++)rk[i]=tp[i]=0;
  for(int i=0;i<=m;i++)cnt[i]=0;
  for(int i=1;i<=n;i++)cnt[rk[i]=s[i]]++;
  for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
  for(int i=n;i>=1;i--)sa[cnt[rk[i]]--]=i;
  for(int w=1;;w<<=1){
    // tp: 按第二关键字排好的后缀
    int p=0;
    for(int i=n-w+1;i<=n;i++)tp[++p]=i;
    for(int i=1;i<=n;i++)if(sa[i]>w)tp[++p]=sa[i]-w;
    // 按第一关键字 rk 做稳定计数排序
    for(int i=0;i<=m;i++)cnt[i]=0;
    for(int i=1;i<=n;i++)cnt[rk[i]]++;
    for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
    for(int i=n;i>=1;i--)sa[cnt[rk[tp[i]]]--]=tp[i];
    // tp 改存旧的 rk, 重新编号
    for(int i=1;i<=n;i++)tp[i]=rk[i];
    p=0;
    for(int i=1;i<=n;i++){
      int x=sa[i],y=sa[i-1];
      bool same=i>1&&tp[x]==tp[y]&&tp[x+w]==tp[y+w];
      rk[x]=same?p:++p;
    }
    if(p==n)break;
    m=p;
  }
}
void getheight(char*s,int n){
  for(int i=1,k=0;i<=n;i++){
    if(rk[i]==1){height[1]=k=0;continue;}
    if(k)k--;
    int j=sa[rk[i]-1];
    while(i+k<=n&&j+k<=n&&s[i+k]==s[j+k])k++;
    height[rk[i]]=k;
  }
}
```

SuffixAutomaton.h

Description: 后缀自动机 (SAM). 根为 1, 节点数至多 2n-1. len[v] 为状态 v 中最长串的长度, fa[v] 为后缀链接, ch[v][c] 为转移. siz[v] 为 v 的 endpos 集合大小, 即 v 中每个串的出现次数 (calc 后有效). calc 按 len 计数排序得到拓扑序 ord, 逆序把 siz 累加到 fa, 并返回本质不同子串个数 $\sum_{v>1} len_v - len_{fa_v}$.
Usage: `init(); // 多组数据先清空`
`for(int i=1;i<=n;i++)extend(s[i]-'a');`
`ll d=calc(); // 本质不同子串数, 同时求出 siz`
`// t 的出现次数: 从 1 沿 ch 走 t, 走不动为 0, 否则为 siz[u]`
Memory: $\mathcal{O}(n\Sigma)$
Time: $\mathcal{O}(n\Sigma)$

7531e2, 40 lines

```
const int N=5e5+5; // 串长上限
int len[N*2],fa[N*2],ch[N*2][26],siz[N*2],tot=1,last=1;
int buc[N],ord[N*2];
void init(){
  for(int i=1;i<=tot;i++)
    len[i]=fa[i]=siz[i]=0,memset(ch[i],0,sizeof ch[i]);
  tot=last=1;
}
void extend(int c){
  int p=last,cur=++tot;
  len[cur]=len[p]+1;siz[cur]=1;
  for(;p&&!ch[p][c];p=fa[p])ch[p][c]=cur;
  if(!p)fa[cur]=1;
  else{
    int q=ch[p][c];
    if(len[q]==len[p]+1)fa[cur]=q;
    else{
      int cl=++tot; // 复制 q, 其 siz 为 0
      len[cl]=len[p]+1;fa[cl]=fa[q];
      memcpy(ch[cl],ch[q],sizeof ch[q]);
      for(;p&&ch[p][c]==q;p=fa[p])ch[p][c]=cl;
      fa[q]=fa[cur]=cl;
    }
  }
  last=cur;
}
ll calc(){
  int n=len[last];
  for(int i=0;i<=n;i++)buc[i]=0;
  for(int i=1;i<=tot;i++)buc[len[i]]++;
  for(int i=1;i<=n;i++)buc[i]+=buc[i-1];
  for(int i=tot;i>=1;i--)ord[buc[len[i]]--]=i;
  ll res=0;
  for(int i=tot;i>=2;i--){ // ord[1] 为根
    int v=ord[i];
    siz[fa[v]]+=siz[v];
    res+=len[v]-len[fa[v]];
  }
  return res;
}
```

Hashing.h

Description: 字符串哈希, 双模数. 串 s[1..n] 为 1-indexed. 两个模数的结果合成一个 ll 返回, 可直接比较或放进 map/set. 双模冲突概率约 10^{-18}, 很安全. 单模 10^9+7 在比较约 10^5 个串时 就可能因生日悖论冲突; 自然溢出 (unsigned long long) 最快, 但会被 Thue-Morse 串卡掉. 怕被 hack 时可把 B1, B2 换成随机数.
Usage: `build(s,n); // 预处理 s 的前缀哈希`
`ll x=get(l,r); // s[l..r] 的哈希`
`ll y=calc(t,m); // 整个 t[1..m] 的哈希, 可与 get 比较`
Time: 预处理 $\mathcal{O}(n)$, 查询 $\mathcal{O}(1)$

e616c9, 23 lines

```
const int N=1e6+5; // 串长上限
const int M1=1e9+7,M2=998244353,B1=131,B2=13331;
int h1[N],h2[N],p1[N],p2[N];
void build(char*s,int n){
  p1[0]=p2[0]=1;h1[0]=h2[0]=0;
  for(int i=1;i<=n;i++){
    p1[i]=1ll*p1[i-1]*B1%M1;
    p2[i]=1ll*p2[i-1]*B2%M2;
    h1[i]=(1ll*h1[i-1]*B1+s[i])%M1;
    h2[i]=(1ll*h2[i-1]*B2+s[i])%M2;
  }
}
ll get(int l,int r){
  ll x=(h1[r]-1ll*h1[l-1]*p1[r-l+1]%M1+M1)%M1;
  ll y=(h2[r]-1ll*h2[l-1]*p2[r-l+1]%M2+M2)%M2;
  return x*M2+y;
}
ll calc(char*t,int m){
  ll x=0,y=0;
  for(int i=1;i<=m;i++)
    x=(x*B1+t[i])%M1,y=(y*B2+t[i])%M2;
  return x*M2+y;
}
```

AhoCorasick.h

Description: AC 自动机. 根为 0, 字符集为小写字母. build 用 BFS 求 fail, 并把缺失的转移补成 tr[fail[u]][c] (Trie 图). query 统计文本中每个模式串的出现次数: 先在每个经过的节点 cnt++, 再按 BFS 逆序沿 fail 树累加, 节点 v 的 cnt 即以 v 为后缀的出现次数. 允许重复模式串, 不允许空串.
Usage: `init(); // 多组数据先清空`
`// 第 i 个模式串 p[i][1..len[i]]`
`for(int i=1;i<=k;i++)insert(p[i],len[i],i);`
`build();query(t,m); // 第 i 个模式串出现次数: cnt[pos[i]]`
`// 换一个文本再 query 前, 先把 cnt[0..tot] 清 0`
Time: 构建 $\mathcal{O}(26L)$, L 为模式串总长; 查询 $\mathcal{O}(m+L)$

f8ac5d, 37 lines

```
const int N=1e6+5; // 模式串总长上限
int tr[N][26],fail[N],cnt[N],q[N],tot;
```

```
int pos[N]; // pos[i]: 第 i 个模式串结尾的节点
void init(){
  for(int i=0;i<=tot;i++)
    memset(tr[i],0,sizeof tr[i]),fail[i]=cnt[i]=0;
  tot=0;
}
void insert(char*s,int n,int id){
  int u=0;
  for(int i=1;i<=n;i++){
    int c=s[i]-'a';
    if(!tr[u][c])tr[u][c]=++tot;
    u=tr[u][c];
  }
  pos[id]=u;
}
void build(){
  int hd=1,tl=0;
  for(int c=0;c<26;c++)if(tr[0][c])q[++tl]=tr[0][c];
  while(hd<=tl){
    int u=q[hd++];
    for(int c=0;c<26;c++){
      int&v=tr[u][c];
      if(v)fail[v]=tr[fail[u]][c],q[++tl]=v;
      else v=tr[fail[u]][c];
    }
  }
}
void query(char*t,int m){
  for(int i=1,u=0;i<=m;i++){
    u=tr[u][t[i]-'a'];
    cnt[u]++;
  }
  // q[1..tot] 为 BFS 序, 逆序累加到 fail
  for(int i=tot;i>=1;i--)cnt[fail[q[i]]]+=cnt[q[i]];
}
```

杂项 (11)

11.1 区间

IntervalContainer.h

Description: 用 set 维护若干互不相交的整数闭区间 $[l,r]$, 相交或相邻的区间 会自动合并. add 加入 $[l,r]$, del 删掉 $[l,r]$ 内所有的点 (可能把一个区间切成两半). tot 为被覆盖的点数.
Usage: `add(1, 5); add(7, 9); add(6, 6); // st = {[1, 9]}`
`del(3, 4); // st = {[1, 2], [5, 9]}, tot = 7`
`auto it = getIt(x); // 包含 x 的区间, 不存在时为 st.end()`
`// 多组数据: st.clear(), tot = 0`
Time: 均摊 $\mathcal{O}(\log N)$
a19759, 27 lines

```
set<pair<int,int>>st;
ll tot; // 被覆盖的点数
void add(int l,int r){
  auto it=st.lower_bound({l,INT_MIN});
  if(it!=st.begin()&&prev(it)->second>=l-1)it--;
  while(it!=st.end()&&it->first<=r+1){
    l=min(l,it->first),r=max(r,it->second);
    tot-=it->second-it->first+1;
    it=st.erase(it);
  }
  st.insert({l,r}),tot+=r-l+1;
}
void del(int l,int r){
  auto it=st.lower_bound({l,INT_MIN});
  if(it!=st.begin()&&prev(it)->second>=l)it--;
  while(it!=st.end()&&it->first<=r){
    auto[L,R]=*it;
    it=st.erase(it),tot-=R-L+1;
    if(L<l)st.insert({L,l-1}),tot+=l-L;
    if(R>r)st.insert({r+1,R}),tot+=R-r;
  }
}
set<pair<int,int>>::iterator getIt(int x){
  auto it=st.upper_bound({x,INT_MAX});
  if(it==st.begin()||prev(it)->second<x)return st.end();
  return prev(it);
}
```

11.2 其他算法

TernarySearch.h

Description: 三分. 整数版 tern 在 $[l,r]$ 中找 f 的最大值点, 要求 f 先严格上升, 到最大值后不增 (二分 $f(mid) < f(mid+1)$). 返回最大值点中最左的. 求最小值把 $<$ 改成 $>$. 若有平台不能用三分. 实数版 ternR 对单峰的 F 三分 100 次.
Usage: `ll f(int x){...} // 自己写, 全局函数`
`int p = tern(1, n); // f(p) 为最大值`
`double x = ternR(lo, hi); // 实数版, F(x) 最大`
Time: $\mathcal{O}(\log(r-l))$ 次求值
691d4b, 18 lines

```
ll f(int x){return -(ll)(x-3)*(x-3);} // 按题目改
int tern(int l,int r){
  while(l<r){
    int mid=((ll)l+r)>>1;
    if(f(mid)<f(mid+1))l=mid+1; // 最大值在 mid 右边
    else r=mid;
  }
  return l;
}
double F(double x){return -(x-1)*(x-1);} // 按题目改
double ternR(double l,double r){
  for(int it=0;it<100;it++){
    double m1=l+(r-l)/3,m2=r-(r-l)/3;
    if(F(m1)<F(m2))l=m1;
    else r=m2;
  }
  return l;
}
```

LIS.h

Description: 最长严格上升子序列, 返回一个最长的子序列的下标 (递增). d[k] 为长度 k 的上升子序列的最小结尾, pos[k] 为它的下标, pre 记前驱. 最长不降子序列: 把 lower_bound 改成 upper_bound.
Usage: `// 读入 n 和 a[1..n]`
`vector<int> id = lis(n); // 值为 a[id[i]]`
`// 多组数据不用清空`
Time: $\mathcal{O}(N \log N)$
b6c906, 14 lines

```
const int N=2e5+5;
int a[N],d[N],pos[N],pre[N]; // a 的类型可以改成 ll, d 也要改
vector<int>lis(int n){
  int len=0;
  for(int i=1;i<=n;i++){
    int k=lower_bound(d+1,d+len+1,a[i])-d;
    d[k]=a[i],pos[k]=i,pre[i]=pos[k-1];
    len=max(len,k);
  }
  vector<int>res;
  for(int i=pos[len];i;i=pre[i])res.push_back(i);
  reverse(res.begin(),res.end());
  return res;
}
```

11.3 动态规划

KnuthDP.h

Description: 区间 DP 的 Knuth 优化 (四边形不等式). $f[l][r] = \min_{l\le k<r}(f[l][k]+f[k+1][r]) + w(l,r)$. 若对 $a \le b \le c \le d$ 有 $w(a,c)+w(b,d) \le w(a,d)+w(b,c)$ 且 $w(b,c) \le w(a,d)$, 则最优分割点 $opt[l][r-1] \le opt[l][r] \le opt[l+1][r]$, 只在这个范围里枚举 k, 总复杂度 $O(n^2)$. 例子是石子合并, w 为区间和.
Usage: `// 读入 n, s[i] = s[i-1] + a[i]; 按题目改 w`
`ll ans = knuth(n); // f[1][n]`
Memory: N = 3005 时约 110MB
Time: $\mathcal{O}(N^2)$
7df0ce, 17 lines

```
const int N=3005;
ll f[N][N],s[N];
int opt[N][N]; // f[l][r] 的最优分割点
ll w(int l,int r){return s[r]-s[l-1];} // 按题目改
ll knuth(int n){
  for(int i=1;i<=n;i++)f[i][i]=0,opt[i][i]=i;
  for(int len=2;len<=n;len++)
    for(int l=1,r=len;r<=n;l++,r++){
      f[l][r]=LLONG_MAX;
      int R=min(r-1,opt[l+1][r]);
      for(int k=opt[l][r-1];k<=R;k++){
        ll v=f[l][k]+f[k+1][r]+w(l,r);
        if(v<f[l][r])f[l][r]=v,opt[l][r]=k;
      }
    }
  return f[1][n];
}
```

DivideAndConquerDP.h

Description: 分治优化 DP. 把 $1..n$ 分成 m 段 (每段非空), 段 $[l,r]$ 的代价为 $w(l,r)$: $f_j[i] = \min_{k<i} f_{j-1}[k] + w(k+1,i)$. 若最优的 k 随 i 单调不减 (w 满足四边形不等式时成立), 先求中点 mid 的最优 k, 左半边的 k 不超过它, 右半边不小于它. g 为上一层, f 为当前层.
Usage: `// 读入 n, a[i], 按题目改 w`
`ll ans = solveDP(n, m); // 分成 m 段的最小代价, m 不超过 n`
Time: $\mathcal{O}(mN \log N)$
9dd5ce, 22 lines

```
const int N=1e5+5;
ll f[N],g[N],s[N];
ll w(int l,int r){ll x=s[r]-s[l-1];return x*x;} // 按题目改
// 求 f[l..r], 它们的最优 k 在 [kl, kr] 内
void dc(int l,int r,int kl,int kr){
  if(l>r)return;
  int mid=(l+r)/2,best=kl;
  f[mid]=LLONG_MAX;
  for(int k=kl;k<=min(mid-1,kr);k++){
    ll v=g[k]+w(k+1,mid);
    if(v<f[mid])f[mid]=v,best=k;
  }
  dc(l,mid-1,kl,best),dc(mid+1,r,best,kr);
}
ll solveDP(int n,int m){
  for(int i=1;i<=n;i++)g[i]=w(1,i); // 第 1 层
  for(int j=2;j<=m;j++){
    dc(j,n,j-1,n-1);
    for(int i=j;i<=n;i++)g[i]=f[i];
  }
  return g[n];
}
```

WQS 二分 (Aliens trick)

适用于恰好选 k 个或恰好分 k 段的最优值 $g(k)$, 且 g 关于 k 是凸

函数 (求最小值时下凸, 求最大值时上凸). 凸性一般靠打表观察, 或由四边形不等式、费用流模型得到.

- **去掉个数限制**: 每选一个额外加代价 λ, 做不限个数的 DP, 求 $h(\lambda)=\min_k\{g(k)+\lambda k\}$, 同时记录取到最优时选了几个, 记为 $cnt(\lambda)$.
- **打平手时取最少个数**: DP 里值相同的转移一律选个数更少的. 这样 $cnt(\lambda)$ 随 λ 单调不增.
- **二分**: 找最小的 λ 使 $cnt(\lambda)\le k$, 答案是 $g(k)=h(\lambda)-\lambda k$. 注意减去的是 λk 而不是 $\lambda\cdot cnt$, 这样多个 k 共线时也正确.
- **求最大值**: 每选一个减去 λ, $h(\lambda)=\max_k\{g(k)-\lambda k\}$, 同样平手取最少个数, 找最小的 λ 使 $cnt(\lambda)\le k$, 答案 $h(\lambda)+\lambda k$.
- **细节**: g 是整数时 λ 只需整数二分, 范围取相邻差分的最小到最大值 (如 $[-10^{12},10^{12}]$). 总复杂度是 DP 的复杂度乘 $\log V$.
- **常见题**: 恰好 k 段划分, 恰好选 k 个不相邻元素, 恰好 k 条白边的最小生成树.

11.4 Kruskal 重构树

用于 只经过边权不超过 x 的边 能到达哪些点, 以及 两点间所有路径中最大边权的最小值 (最小瓶颈路) 这类问题.

- **建树**: 边按权从小到大排序做 Kruskal. 每合并两个不同的连通块 (并查集的根为 a,b) 时, 新建一个点 z (编号从 $n+1$ 开始), 点权 $val[z]=w$, 让 z 成为 a 和 b 的父亲, 并查集里也把 a,b 并到 z 上. 原图的点全是叶子, 连通图建完共 $2n-1$ 个点, 根是最后新建的点.
- **性质**: 二叉树, 且从叶子往根走点权不降 (祖先的点权不小于子孙).
- **最小瓶颈路**: 原图中 u,v 之间所有路径上最大边权的最小值等于 $val[\mathrm{lca}(u,v)]$. 不连通时两点不在同一棵树里.
- **只走边权不超过 x 的边**: 从 u 往上跳, 跳到点权不超过 x 的最高祖先 a (倍增), 能到达的点就是 a 子树里的所有叶子. 用 dfs 序把子树变成区间, 就能配合主席树等求这些点里的第 k 大等.
- **反过来**: 若关心路径上最小边权的最大值, 改成边权从大到小建树, 上面的不等号全部反过来.

11.5 调试技巧

- `signal(SIGSEGV, [](int) { _Exit(0); });` 把段错误变成 Wrong Answer. 同理可捕获 SIGABRT (断言失败) 和 SIGFPE (除零). `_GLIBCXX_DEBUG` 报错时产生 SIGABRT (gcc 5.4.0 上似乎是 SIGSEGV).
- `feenableexcept(29);` 在遇到 NaN (1), 除零 (4), 无穷 (8) 和非规格化数 (16) 时终止程序.

11.6 优化技巧

`__builtin_ia32_ldmxcsr(40896);` 关闭非规格化数 (它们在最小值附近让浮点运算慢 20 倍).

11.6.1 位运算技巧

- `x&-x` 是 `x` 的最低位的 1.
- `c=x&-x,r=x+c;(((r^x)>>2)/c)|r` 是比 `x` 大且 1 的个数相同的下一个数.
- 子集和 (SOS DP):

```
for(int b=0;b<K;b++)
 for(int i=0;i<(1<<K);i++)
  if(i>>b&1)D[i]+=D[i^(1<<b)];
```

- 枚举所有集合的所有子集, 总共 3^n 次. 只要非空子集时内层写成 `for(int t=s;t;t=(t-1)&s):`

```
for(int s=0;s<(1<<n);s++) // 每个集合 s
  for(int t=s;;t=(t-1)&s){ // s 的每个子集 t, 含空集
    // 处理 (s, t)
    if(!t)break;
  }
```

11.6.2 bitset 加速 pragma (放在 bits/stdc++.h 之前)

- `#pragma GCC optimize("O3,unroll-loops")`
- `#include <bits/allocator.h>`
- `#pragma GCC target("avx2,bmi,bmi2,popcnt,lzcnt")`

UniformRandom.h

Description: 随机数. mt19937_64 用时间做种子 (防 hack), 不要用 rand().
Usage: `ll x = rnd(1, 100); // [1, 100] 内均匀随机整数`
`shuffle(a + 1, a + n + 1, rng); // 随机打乱 a[1..n]`
`double y = uniform_real_distribution<double>(0, 1)(rng);`
Time: $\mathcal{O}(1)$

bb326d, 5 lines

```
mt19937_64 rng(chrono::steady_clock::now()
  .time_since_epoch().count());
ll rnd(ll l,ll r){ // [l, r] 内均匀随机
  return uniform_int_distribution<ll>(l,r)(rng);
}
```

StressTest.sh

24 lines

```
# Usage: bash stress.sh [fast] [slow] [gen] [# iters]
# Example: bash stress.sh main main2 generator 1000
# remove this and add -O2 if you want, cc8b47
flags="-std=c++20 -g -fsanitize=address,undefined"
g++ $flags $1.cpp -o $1
g++ $flags $2.cpp -o $2
g++ $flags $3.cpp -o $3
for ((t=1; t<=$4; t++)) do
  ./$3 > input
  ./$2 < input > outslow
  ./$1 < input > outwrong
  if [[ "$?" -ne 0 ]] || ! (cmp -s "outwrong" "outslow") then
    echo error found
    echo input:
    cat input
    echo wrong output:
    cat outwrong
    echo slow output:
    cat outslow
    exit
  fi
  echo passed $t tests
done
echo passed all tests
```